

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №10
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии Воронкин Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Сегментация строительных изображений с помощью U-Net (Keras и PyTorch). 5 и 7 классов. Сравнение экспериментов.

Репозиторий: https://github.com/bickrosss/NN_lab10

Задание Lite: 5 классов, 10 экспериментов.

1.1.1. Импорт библиотек.

Импортированы библиотеки TensorFlow/Keras, matplotlib, numpy, PIL, sklearn и вспомогательные модули для работы с файлами и памятью.

```
# Импортируем модели keras: Model
from tensorflow.keras.models import Model

# Импортируем стандартные слои keras
from tensorflow.keras.layers import Input, Conv2DTranspose, concatenate, Activation
from tensorflow.keras.layers import MaxPooling2D, Conv2D, BatchNormalization, UpSampling2D

# Импортируем оптимизатор Adam
from tensorflow.keras.optimizers import Adam

# Импортируем модуль pyplot библиотеки matplotlib для построения графиков
import matplotlib.pyplot as plt

# Импортируем модуль image для работы с изображениями
from tensorflow.keras.preprocessing import image

# Импортируем библиотеку numpy
import numpy as np

# Импортируем метод деления выборки
from sklearn.model_selection import train_test_split

# загрузка файлов по HTML ссылке
import gdown

# Для работы с файлами
import os

# Для генерации случайных чисел
import random

import time

# импортируем модель Image для работы с изображениями
from PIL import Image

# очистка ОЗУ
import gc
```

Рисунок 1 – Импорт библиотек

1.1.2. Загрузка датасета и глобальные параметры.

Датасет construction_256x192 загружен из облака (214 МБ). Параметры: IMG_WIDTH=192, IMG_HEIGHT=256. NUM_CLASSES изменён с 16 до 5.

```

# Загрузка датасета из облака

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/114/construction_256x192.zip', None, quiet=False)
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/114/construction_512x384.zip', None, quiet=False)

!unzip -q 'construction_256x192.zip' # распаковываем архив

Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/114/construction_256x192.zip
To: /content/construction_256x192.zip
100%|██████████| 214M/214M [00:03<00:00, 60.6MB/s]

# Глобальные параметры

IMG_WIDTH = 192          # Ширина картинки
IMG_HEIGHT = 256         # Высота картинки
NUM_CLASSES = 16         # Задаем количество классов на изображении
TRAIN_DIRECTORY = 'train' # Название папки с файлами обучающей выборки
VAL_DIRECTORY = 'val'     # Название папки с файлами проверочной выборки

```

Рисунок 2 – Загрузка датасета и глобальные параметры

1.1.3. Загрузка изображений.

Загружены оригинальные изображения (1900 обучающих, 100 проверочных) и сегментированные маски из соответствующих директорий.

```

train_images = [] # Создаем пустой список для хранения оригинальных изображений обучающей выборки
val_images = [] # Создаем пустой список для хранения оригинальных изображений проверочной выборки

cur_time = time.time() # Засекаем текущее время

# Проходим по всем файлам в каталоге по указанному пути
for filename in sorted(os.listdir(TRAIN_DIRECTORY+'original')):
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    train_images.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'original',filename),
        target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок обучающей выборки
print ('Обучающая выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в обучающей выборке
print ('Количество изображений: ', len(train_images))

cur_time = time.time() # Засекаем текущее время

# Проходим по всем файлам в каталоге по указанному пути
for filename in sorted(os.listdir(VAL_DIRECTORY+'original')):
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    val_images.append(image.load_img(os.path.join(VAL_DIRECTORY+'original',filename),
        target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок проверочной выборки
print ('Проверочная выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в проверочной выборке
print ('Количество изображений: ', len(val_images))

Обучающая выборка загружена. Время загрузки: 0.37с
Количество изображений: 1900
Проверочная выборка загружена. Время загрузки: 0.02с
Количество изображений: 100

```

Рисунок 3 – Загрузка оригинальных изображений

```

train_segments = [] # Создаем пустой список для хранения оригинальных изображений обучающей выборки
val_segments = [] # Создаем пустой список для хранения оригинальных изображений проверочной выборки

cur_time = time.time() # Засекаем текущее время

for filename in sorted(os.listdir(TRAIN_DIRECTORY+'/segment')): # Проходим по всем файлам в каталоге по указанному пути
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    train_segments.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'/segment',filename),
        target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок обучающей выборки
print ('Обучающая выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в обучающем наборе сегментированных изображений
print ('Количество изображений: ', len(train_segments))

cur_time = time.time() # Засекаем текущее время

for filename in sorted(os.listdir(VAL_DIRECTORY+'/segment')): # Проходим по всем файлам в каталоге по указанному пути
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    val_segments.append(image.load_img(os.path.join(VAL_DIRECTORY+'/segment',filename),
        target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок проверочной выборки
print ('Проверочная выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в проверочном наборе сегментированных изображений
print ('Количество изображений: ', len(val_segments))

```

```

Обучающая выборка загружена. Время загрузки: 0.35с
Количество изображений: 1900
Проверочная выборка загружена. Время загрузки: 0.02с
Количество изображений: 100

```

Рисунок 4 – Загрузка сегментированных изображений

1.1.4. Подготовка данных: 16 → 5 классов.

Задан маппинг MAP_16_TO_5 (0 – фон, 1 – стены, 2 – проёмы, 3 – крыша, 4 – прочее). Данные хранятся в формате uint8, нормализация 1/255 выполняется внутри модели слоем Rescaling. Распределение классов: фон – 47 798 941, стены – 32 926 001, проёмы – 9 025 507, крыша – 2 931 560, прочее – 706 791. Объём: X – 281.2 МБ, Y – 93.8 МБ.

```

) # 1. ФИНАЛЬНАЯ ПОДГОТОВКА ДАННЫХ: 16 -> 5 классов
# Ключевая оптимизация памяти:
# * X храним как uint8 (а не float32) – в 4 раза меньше
# * Y храним как карту классов int8 (N, H, W) – в 20 раз меньше, чем one-hot float32
# * нормализация 1/255 будет внутри модели слоем Rescaling
# * лосс будет sparse_categorical_crossentropy – one-hot не нужен

import numpy as np
import gc
import tensorflow as tf

NUM_CLASSES = 5 # было 16, по заданию делаем 5

# Цвета 16 исходных классов (RGB)
# Стандартная палитра датасета construction_256x192.
# Если ваши маски используют другие цвета – встроенный авто-детектор ниже
# найдёт фактические цвета в данных и сообщит об этом.
CLASS_COLORS_16 = [
    (0, 0, 0), # 0 - фон
    (128, 0, 0), # 1
    (0, 128, 0), # 2
    (128, 128, 0), # 3
    (0, 0, 128), # 4
    (128, 0, 128), # 5
    (0, 128, 128), # 6
    (128, 128, 128), # 7
    (64, 0, 0), # 8
    (192, 0, 0), # 9
    (64, 128, 0), # 10
    (192, 128, 0), # 11
    (64, 0, 128), # 12
    (192, 0, 128), # 13
    (64, 128, 128), # 14
    (192, 128, 128), # 15
]

# Мappings 16 -> 5 укрупнённых классов
# 0=фон, 1=стены, 2=проёмы, 3=крыша, 4=прочее
MAP_16_TO_5 = np.array([
    0,
    1, 1, 1,
    2, 2, 2, 2,
    3, 3, 3, 3,
    4, 4, 4, 4,
], dtype=np.int8)

def build_color_lut():
    """Хэш-таблица: RGB-цвет (24 бита) -> класс 0..15."""
    lut = np.full(256 ** 3, 255, dtype=np.uint8) # 255 = неизвестный
    for idx, (r, g, b) in enumerate(CLASS_COLORS_16):
        key = (r << 16) | (g << 8) | b
        lut[key] = idx
    return lut

def seg_rgb_to_class5(seg_img_rgb, lut):
    """PIL/np RGB-маска -> карта 5 классов (H, W) int8.

    Использует LUT по 24-битному ключу: один проход, без 16 булевых масок.
    """
    seg = np.asarray(seg_img_rgb, dtype=np.uint8)
    if seg.ndim == 2:
        seg = np.stack([seg] * 3, axis=-1)
    if seg.shape[-1] == 4:
        seg = seg[..., :3]
    r, g, b = seg[..., 0].astype(np.uint32), seg[..., 1].astype(np.uint32), seg[..., 2].astype(np.uint32)
    keys = (r << 16) | (g << 8) | b
    cls16 = lut[keys] # (H, W) uint8 в диапазоне 0..15 (или 255)

```

Рисунок 6 – Финальная подготовка данных: 16 → 5 классов

```

cls16 = np.zeros((H, W)) # (H, W) zeros в диапазоне 0..255 (uint8)
cls16 = np.where(cls16 == 255, 0, cls16) # неизвестные цвета -> фон
cls5 = MAP_16_TO_5[cls16] # (H, W) int8
return cls5

# Готовим X
print('Готовим X (uint8)...')
# Берём первый, чтобы узнать форму
first = np.asarray(train_images[0], dtype=np.uint8)
H, W = first.shape[:2]
print(f' Реальная форма картинки: H={H}, W={W}, channels={first.shape[2] if first.ndim==3 else 1}')

x_train = np.empty((len(train_images), H, W, 3), dtype=np.uint8)
for i, img in enumerate(train_images):
    arr = np.asarray(img, dtype=np.uint8)
    if arr.ndim == 2:
        arr = np.stack([arr]*3, axis=-1)
    if arr.shape[-1] == 4:
        arr = arr[..., :3]
    x_train[i] = arr

x_val = np.empty((len(val_images), H, W, 3), dtype=np.uint8)
for i, img in enumerate(val_images):
    arr = np.asarray(img, dtype=np.uint8)
    if arr.ndim == 2:
        arr = np.stack([arr]*3, axis=-1)
    if arr.shape[-1] == 4:
        arr = arr[..., :3]
    x_val[i] = arr

print(f' x_train: {x_train.shape} dtype={x_train.dtype} ~{x_train.nbytes/1024/1024:.1f} MB')
print(f' x_val: {x_val.shape} dtype={x_val.dtype} ~{x_val.nbytes/1024/1024:.1f} MB')

# Сразу удаляем исходные списки картинок
del train_images, val_images
gc.collect()

# Готовим Y
print('\nГотовим Y (int8 карта классов, 5 классов)...')

# Авто-детектор: если канонических цветов в первой маске не нашлось,
# построим LUT по уникальным цветам, отсортированным по частоте.
LUT = build_color_lut()

probe = np.asarray(train_segments[0], dtype=np.uint8)
if probe.shape[-1] == 4:
    probe = probe[..., :3]
pr, pg, pb = probe[..., 0].astype(np.uint32), probe[..., 1].astype(np.uint32), probe[..., 2].astype(np.uint32)
pk = (pr << 16) | (pg << 8) | pb
known_hits = (LUT[pk] != 255).mean()
print(f' Доля пикселей с известным цветом в первой маске: {known_hits:.3f}')

if known_hits < 0.5:
    # Авто-детект цветов: берём 16 самых частых уникальных цветов по всей train-выборке
    print(' >> Стандартные цвета не подошли, автоматически выявляем 16 классов по частоте...')
    keys_all = []
    for seg in train_segments:
        a = np.asarray(seg, dtype=np.uint8)
        if a.shape[-1] == 4: a = a[..., :3]
        r_, g_, b_ = a[..., 0].astype(np.uint32), a[..., 1].astype(np.uint32), a[..., 2].astype(np.uint32)
        keys_all.append(((r_ << 16) | (g_ << 8) | b_).ravel())
    keys_all = np.concatenate(keys_all)
    vals, counts = np.unique(keys_all, return_counts=True)
    order = np.argsort(-counts)
    top_keys = vals[order][:16]
    print(f' Найдено уникальных цветов: {len(vals)}; берём топ-16.')
    LUT = np.full(256 * 3, 255, dtype=np.uint8)
    for idx, k in enumerate(top_keys):
        LUT[k] = idx

```

Рисунок 7 – Формирование тензоров X и Y

```

        for idx, k in enumerate(top_keys):
            LUT[k] = idx
        del keys_all, vals, counts, order, top_keys
        gc.collect()

y_train = np.empty((len(train_segments), H, W), dtype=np.int8)
for i, seg in enumerate(train_segments):
    y_train[i] = seg_rgb_to_class5(seg, LUT)

y_val = np.empty((len(val_segments), H, W), dtype=np.int8)
for i, seg in enumerate(val_segments):
    y_val[i] = seg_rgb_to_class5(seg, LUT)

print(f' y_train: {y_train.shape} dtype={y_train.dtype} ~{y_train.nbytes/1024/1024:.1f} MB')
print(f' y_val: {y_val.shape} dtype={y_val.dtype} ~{y_val.nbytes/1024/1024:.1f} MB')

# Распределение классов
uniq, cnts = np.unique(y_train, return_counts=True)
print(f' Распределение классов в y_train: {dict(zip(uniq.tolist(), cnts.tolist()))}')

# Удаляем исходные сегменты
del train_segments, val_segments
gc.collect()

print('\nГотово. Память входных тензоров:')
print(f' X: {(x_train.nbytes + x_val.nbytes)/1024/1024:.1f} MB')
print(f' Y: {(y_train.nbytes + y_val.nbytes)/1024/1024:.1f} MB')

```

```

.. Готовим X (uint8)...
    Реальная форма картинки: H=192, W=256, channels=3
    x_train: (1900, 192, 256, 3) dtype=uint8 ~267.2 MB
    x_val: (100, 192, 256, 3) dtype=uint8 ~14.1 MB

    Готовим Y (int8 карта классов, 5 классов)...
    Доля пикселей с известным цветом в первой маске: 0.000
    >> Стандартные цвета не подошли, автоматически выявляем 16 классов по частоте...
    Найдено уникальных цветов: 27; берём top-16.
    y_train: (1900, 192, 256) dtype=int8 ~89.1 MB
    y_val: (100, 192, 256) dtype=int8 ~4.7 MB
    Распределение классов в y_train: {0: 47798941, 1: 32926001, 2: 9025507, 3: 2931560, 4: 706791}

    Готово. Память входных тензоров:
    X: 281.2 MB
    Y: 93.8 MB

```

Рисунок 8 – Результаты формирования тензоров

1.1.5. Модель U-Net.

Модель U-Net с энкодером (3 блока Conv2D + MaxPooling), буттлнеком и декодером (Conv2DTranspose + конкатенация). Параметры filters, kernel_size и функция активации варьируются в экспериментах. Слой Rescaling(1/255) встроен в модель. Выход – sparse softmax, функция потерь – sparse_categorical_crossentropy.


```

# 2. ФУНКЦИЯ ПОСТРОЕНИЯ U-Net (с настраиваемыми параметрами)
from tensorflow.keras.layers import Rescaling

def build_unet(img_h, img_w,
               num_classes=NUM_CLASSES,
               filters=16,
               kernel_size=3,
               activation='relu'):
    """U-Net с настраиваемыми filters / kernel_size / activation в скрытых слоях.

    - Rescaling(1/255) встроен в модель => подаём uint8 напрямую.
    - На выходе sparse softmax + sparse_categorical_crossentropy.
    """
    inp = Input(shape=(img_h, img_w, 3), dtype='uint8')
    x = Rescaling(1.0 / 255.0)(inp)

    # Encoder
    c1 = Conv2D(filters, kernel_size, padding='same')(x); c1 = BatchNormalization()(c1); c1 = Activation(activation)(c1)
    c1 = Conv2D(filters, kernel_size, padding='same')(c1); c1 = BatchNormalization()(c1); c1 = Activation(activation)(c1)
    p1 = MaxPooling2D()(c1)

    c2 = Conv2D(filters*2, kernel_size, padding='same')(p1); c2 = BatchNormalization()(c2); c2 = Activation(activation)(c2)
    c2 = Conv2D(filters*2, kernel_size, padding='same')(c2); c2 = BatchNormalization()(c2); c2 = Activation(activation)(c2)
    p2 = MaxPooling2D()(c2)

    c3 = Conv2D(filters*4, kernel_size, padding='same')(p2); c3 = BatchNormalization()(c3); c3 = Activation(activation)(c3)
    c3 = Conv2D(filters*4, kernel_size, padding='same')(c3); c3 = BatchNormalization()(c3); c3 = Activation(activation)(c3)
    p3 = MaxPooling2D()(c3)

    # Bottleneck
    b = Conv2D(filters*8, kernel_size, padding='same')(p3); b = BatchNormalization()(b); b = Activation(activation)(b)
    b = Conv2D(filters*8, kernel_size, padding='same')(b); b = BatchNormalization()(b); b = Activation(activation)(b)

    # Decoder
    u3 = Conv2DTranspose(filters*4, 2, strides=2, padding='same')(b); u3 = concatenate([u3, c3])
    d3 = Conv2D(filters*4, kernel_size, padding='same')(u3); d3 = BatchNormalization()(d3); d3 = Activation(activation)(d3)
    d3 = Conv2D(filters*4, kernel_size, padding='same')(d3); d3 = BatchNormalization()(d3); d3 = Activation(activation)(d3)

    u2 = Conv2DTranspose(filters*2, 2, strides=2, padding='same')(d3); u2 = concatenate([u2, c2])
    d2 = Conv2D(filters*2, kernel_size, padding='same')(u2); d2 = BatchNormalization()(d2); d2 = Activation(activation)(d2)
    d2 = Conv2D(filters*2, kernel_size, padding='same')(d2); d2 = BatchNormalization()(d2); d2 = Activation(activation)(d2)

    u1 = Conv2DTranspose(filters, 2, strides=2, padding='same')(d2); u1 = concatenate([u1, c1])
    d1 = Conv2D(filters, kernel_size, padding='same')(u1); d1 = BatchNormalization()(d1); d1 = Activation(activation)(d1)
    d1 = Conv2D(filters, kernel_size, padding='same')(d1); d1 = BatchNormalization()(d1); d1 = Activation(activation)(d1)

    out = Conv2D(num_classes, 1, activation='softmax')(d1)

    model = Model(inputs=inp, outputs=out)
    model.compile(
        optimizer=Adam(learning_rate=1e-3),
        loss='sparse_categorical_crossentropy', # у хранится как карта классов
        metrics=['accuracy'],
    )
    return model

```

Рисунок 9 – Функция построения U-Net

1.1.6. Проведение 10 экспериментов (7 эпох).

Проведено 10 экспериментов с фиксированным числом эпох EPOCHS=7 и BATCH_SIZE=8. В каждом эксперименте изменялись filters (8, 16, 32), kernel_size (3, 5, 7) и функция активации (relu, elu, selu, linear).


```

# 3. ЗАПУСК 10 ЭКСПЕРИМЕНТОВ (одинаковое число эпох)
import tensorflow as tf

EPOCHS = 7
BATCH_SIZE = 8

# Размеры берём из реальной формы данных (не из IMG_HEIGHT/IMG_WIDTH).
_, IMG_H, IMG_W, _ = x_train.shape
print(f'Размер входа модели: ({IMG_H}, {IMG_W}, 3)')

# tf.data датасеты: без копирования больших тензоров, с prefetch.
def make_ds(x, y, batch, shuffle):
    ds = tf.data.Dataset.from_tensor_slices((x, y))
    if shuffle:
        ds = ds.shuffle(buffer_size=min(512, len(x)), reshuffle_each_iteration=True)
    return ds.batch(batch).prefetch(tf.data.AUTOTUNE)

train_ds = make_ds(x_train, y_train, BATCH_SIZE, shuffle=True)
val_ds = make_ds(x_val, y_val, BATCH_SIZE, shuffle=False)

experiments = [
    # filters kernel activation (что меняем)
    {'name': 'exp01', 'filters': 16, 'kernel_size': 3, 'activation': 'relu'}, # baseline
    {'name': 'exp02', 'filters': 32, 'kernel_size': 3, 'activation': 'relu'}, # filters ↑
    {'name': 'exp03', 'filters': 8, 'kernel_size': 3, 'activation': 'relu'}, # filters ↓
    {'name': 'exp04', 'filters': 16, 'kernel_size': 5, 'activation': 'relu'}, # kernel ↑
    {'name': 'exp05', 'filters': 16, 'kernel_size': 7, 'activation': 'relu'}, # kernel ↑↑
    {'name': 'exp06', 'filters': 16, 'kernel_size': 3, 'activation': 'elu'}, # activation
    {'name': 'exp07', 'filters': 16, 'kernel_size': 3, 'activation': 'selu'}, # activation
    {'name': 'exp08', 'filters': 16, 'kernel_size': 3, 'activation': 'linear'}, # activation
    {'name': 'exp09', 'filters': 32, 'kernel_size': 5, 'activation': 'elu'}, # комбо
    {'name': 'exp10', 'filters': 8, 'kernel_size': 5, 'activation': 'selu'}, # комбо
]

results = []

for cfg in experiments:
    print('\n' + '=' * 70)
    print(f'{cfg["name"]}: filters={cfg["filters"]}, '
          f'kernel_size={cfg["kernel_size"]}, activation={cfg["activation"]}')
    print('=' * 70)

    # Перед каждой моделью — полная очистка
    tf.keras.backend.clear_session()
    gc.collect()

    model = build_unet(IMG_H, IMG_W,
                      filters=cfg['filters'],
                      kernel_size=cfg['kernel_size'],
                      activation=cfg['activation'])

    history = model.fit(train_ds,
                       validation_data=val_ds,
                       epochs=EPOCHS,
                       verbose=2)

```

Рисунок 10 – Конфигурации 10 экспериментов

```

# Сохраняем только числа, не массивы — экономим ОЗУ
results.append({
    'name': cfg['name'],
    'filters': cfg['filters'],
    'kernel_size': cfg['kernel_size'],
    'activation': cfg['activation'],
    'train_acc': [float(v) for v in history.history['accuracy']],
    'val_acc': [float(v) for v in history.history['val_accuracy']],
    'train_loss': [float(v) for v in history.history['loss']],
    'val_loss': [float(v) for v in history.history['val_loss']],
    'final_val_acc': float(history.history['val_accuracy'][-1]),
    'final_val_loss': float(history.history['val_loss'][-1]),
})

# Очистка между экспериментами
del model, history
tf.keras.backend.clear_session()
gc.collect()

print('\nВсе эксперименты завершены.')

```

Рисунок 11 – Сохранение результатов и очистка памяти

Размер входа модели: (192, 256, 3)

```
=====
exp01: filters=16, kernel_size=3, activation=relu
=====
Epoch 1/7
238/238 - 60s - 251ms/step - accuracy: 0.6468 - loss: 1.0044 - val_accuracy: 0.5136 - val_loss: 1.3061
Epoch 2/7
238/238 - 12s - 51ms/step - accuracy: 0.7289 - loss: 0.7527 - val_accuracy: 0.6385 - val_loss: 1.0562
Epoch 3/7
238/238 - 12s - 52ms/step - accuracy: 0.7583 - loss: 0.6865 - val_accuracy: 0.6148 - val_loss: 1.0681
Epoch 4/7
238/238 - 13s - 53ms/step - accuracy: 0.7712 - loss: 0.6566 - val_accuracy: 0.5924 - val_loss: 1.1015
Epoch 5/7
238/238 - 13s - 54ms/step - accuracy: 0.7805 - loss: 0.6282 - val_accuracy: 0.6233 - val_loss: 0.9965
Epoch 6/7
238/238 - 13s - 55ms/step - accuracy: 0.7891 - loss: 0.6074 - val_accuracy: 0.6433 - val_loss: 0.9904
Epoch 7/7
238/238 - 13s - 55ms/step - accuracy: 0.7995 - loss: 0.5805 - val_accuracy: 0.6993 - val_loss: 0.8532

=====
exp02: filters=32, kernel_size=3, activation=relu
=====
Epoch 1/7
238/238 - 85s - 355ms/step - accuracy: 0.6212 - loss: 1.0871 - val_accuracy: 0.5988 - val_loss: 1.1201
Epoch 2/7
238/238 - 27s - 113ms/step - accuracy: 0.7211 - loss: 0.7746 - val_accuracy: 0.5754 - val_loss: 1.1441
Epoch 3/7
238/238 - 28s - 117ms/step - accuracy: 0.7420 - loss: 0.7156 - val_accuracy: 0.6260 - val_loss: 0.9721
Epoch 4/7
238/238 - 27s - 113ms/step - accuracy: 0.7615 - loss: 0.6748 - val_accuracy: 0.6636 - val_loss: 0.8981
Epoch 5/7
238/238 - 27s - 114ms/step - accuracy: 0.7738 - loss: 0.6435 - val_accuracy: 0.6526 - val_loss: 0.9951
Epoch 6/7
238/238 - 27s - 115ms/step - accuracy: 0.7827 - loss: 0.6201 - val_accuracy: 0.6489 - val_loss: 0.9581
Epoch 7/7
238/238 - 27s - 114ms/step - accuracy: 0.7903 - loss: 0.5906 - val_accuracy: 0.6903 - val_loss: 0.8841

=====
exp03: filters=8, kernel_size=3, activation=relu
=====
Epoch 1/7
238/238 - 40s - 169ms/step - accuracy: 0.5895 - loss: 1.1317 - val_accuracy: 0.4738 - val_loss: 1.2711
Epoch 2/7
238/238 - 8s - 32ms/step - accuracy: 0.7089 - loss: 0.8382 - val_accuracy: 0.6375 - val_loss: 1.0013
Epoch 3/7
238/238 - 8s - 32ms/step - accuracy: 0.7425 - loss: 0.7436 - val_accuracy: 0.6136 - val_loss: 1.0380
Epoch 4/7
238/238 - 8s - 32ms/step - accuracy: 0.7625 - loss: 0.6882 - val_accuracy: 0.6711 - val_loss: 0.9149
Epoch 5/7
238/238 - 8s - 32ms/step - accuracy: 0.7705 - loss: 0.6602 - val_accuracy: 0.6684 - val_loss: 0.9400
Epoch 6/7
238/238 - 7s - 31ms/step - accuracy: 0.7779 - loss: 0.6381 - val_accuracy: 0.6263 - val_loss: 0.9896
Epoch 7/7
238/238 - 7s - 31ms/step - accuracy: 0.7826 - loss: 0.6271 - val_accuracy: 0.6908 - val_loss: 0.8599
```

Рисунок 12 – Вывод хода обучения

1.1.7. Визуализация результатов.

Построены кривые val_accuracy и val_loss по эпохам для всех 10 экспериментов, а также столбчатая диаграмма финальной точности. Наилучший результат: exp01 (filters=16, kernel_size=3, relu) – val_acc=0.6993, val_loss=0.8532.

```

# 4. ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ

# (а) Кривые val_accuracy
plt.figure(figsize=(14, 6))
for r in results:
    label = f"{r['name']} f={r['filters']} k={r['kernel_size']} {r['activation']}"
    plt.plot(range(1, EPOCHS + 1), r['val_acc'], marker='o', label=label)
plt.title('Точность на валидации по эпохам (10 экспериментов)')
plt.xlabel('Эпоха'); plt.ylabel('val_accuracy')
plt.grid(True, alpha=0.3)
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', fontsize=9)
plt.tight_layout(); plt.show()

# (б) Кривые val_loss
plt.figure(figsize=(14, 6))
for r in results:
    label = f"{r['name']} f={r['filters']} k={r['kernel_size']} {r['activation']}"
    plt.plot(range(1, EPOCHS + 1), r['val_loss'], marker='o', label=label)
plt.title('Loss на валидации по эпохам (10 экспериментов)')
plt.xlabel('Эпоха'); plt.ylabel('val_loss')
plt.grid(True, alpha=0.3)
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', fontsize=9)
plt.tight_layout(); plt.show()

# (в) Финальная val_accuracy – столбчатая
plt.figure(figsize=(12, 6))
names = [r['name'] for r in results]
finals = [r['final_val_acc'] for r in results]
bars = plt.bar(names, finals, color='steelblue')
plt.title('Финальная точность на валидации по экспериментам')
plt.ylabel('val_accuracy (последняя эпоха)')
plt.xticks(rotation=45)
for bar, v in zip(bars, finals):
    plt.text(bar.get_x() + bar.get_width()/2, v + 0.005,
             f'{v:.3f}', ha='center', fontsize=9)
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout(); plt.show()

# (г) Итоговая таблица
print('\n' + '=' * 80)
print('ИТОГОВАЯ ТАБЛИЦА ЭКСПЕРИМЕНТОВ')
print('=' * 80)
print(f'{<name>:<8>{<filters>:<10>{<kernel_size>:<10>{<activation>:~<14>{<val_acc>:~<12>{<val_loss>:~<12>"}
print('~' * 80)
for r in results:
    print(f"{r['name']}:~<8>{r['filters']}:~<10>{r['kernel_size']}:~<10>~<14>~<12.4f>{r['final_val_acc']}:~<12.4f>{r['final_val_loss']}:~<12.4f>"}
print('~' * 80)

best = max(results, key=lambda r: r['final_val_acc'])
print(f"лучший: {best['name']} – filters={best['filters']}, ~<14>~<12.4f>{best['final_val_acc']}:~<12.4f>{best['final_val_loss']}:~<12.4f>"}

```

Рисунок 13 – Код визуализации результатов

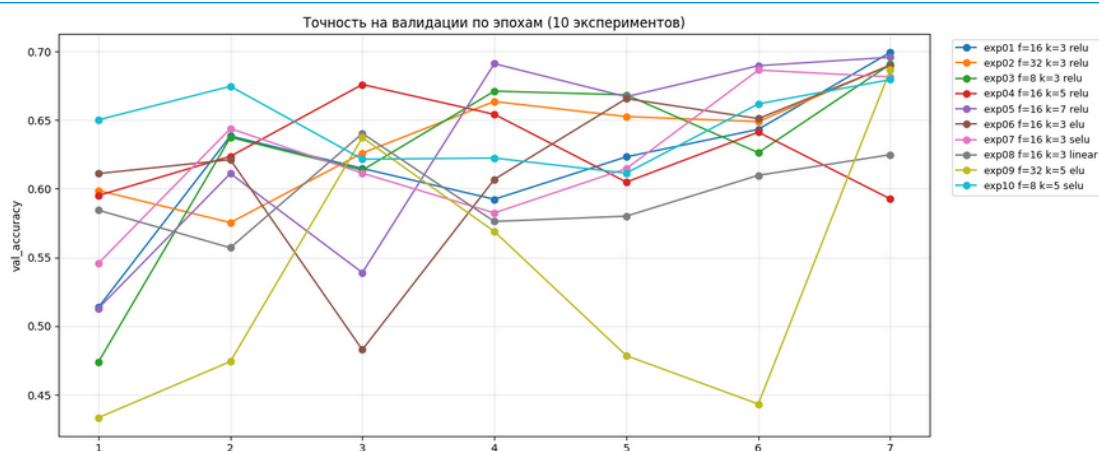


Рисунок 14 – Кривые val_accuracy по эпохам (10 экспериментов, Keras)

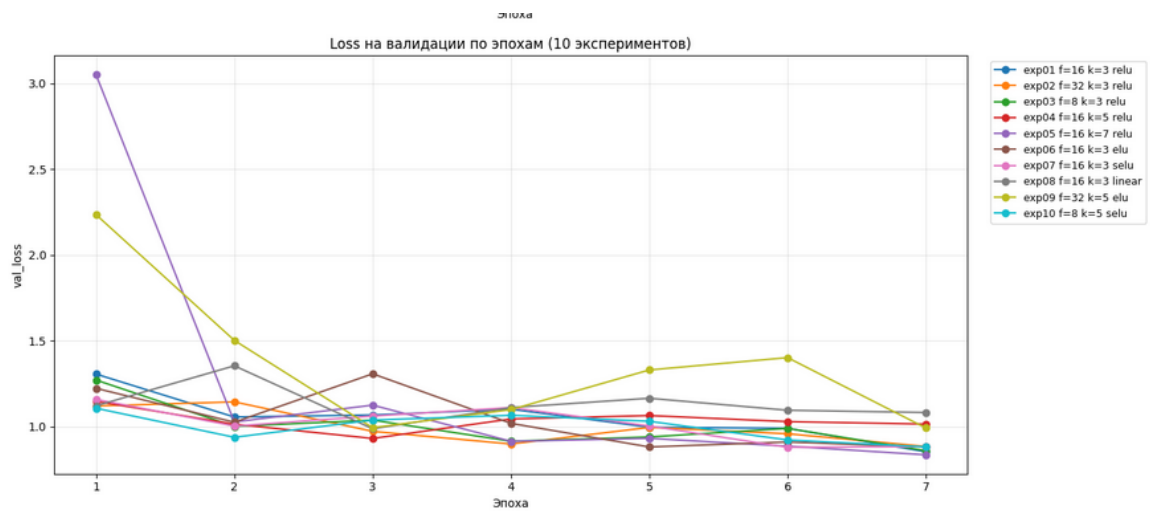


Рисунок 15 – Кривые val_loss по эпохам (10 экспериментов, Keras)



Рисунок 16 – Финальная val_acc по экспериментам

ИТОГОВАЯ ТАБЛИЦА ЭКСПЕРИМЕНТОВ					
name	filters	kernel	activation	val_acc	val_loss
exp01	16	3	relu	0.6993	0.8532
exp02	32	3	relu	0.6903	0.8844
exp03	8	3	relu	0.6908	0.8599
exp04	16	5	relu	0.5926	1.0148
exp05	16	7	relu	0.6958	0.8357
exp06	16	3	elu	0.6898	0.8813
exp07	16	3	selu	0.6815	0.8835
exp08	16	3	linear	0.6248	1.0824
exp09	32	5	elu	0.6866	0.9921
exp10	8	5	selu	0.6795	0.8822

Лучший: exp01 – filters=16, kernel_size=3, activation=relu, val_acc=0.6993

Рисунок 17 – Итоговая таблица экспериментов

1.1.8. Предсказания лучшей модели.

Лучшая модель (exp01: filters=16, kernel_size=3, relu, val_acc=0.6993) переобучена и использована для предсказания масок на 4 случайных изображениях из валидационной выборки.

```
# 5. ВИЗУАЛИЗАЦИЯ ПРЕДСКАЗАНИЙ ЛУЧШЕЙ МОДЕЛИ
import tensorflow as tf
tf.keras.backend.clear_session(); gc.collect()

best_model = build_unet(IMG_H, IMG_W,
                        filters=best['filters'],
                        kernel_size=best['kernel_size'],
                        activation=best['activation'])
best_model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, verbose=2)

# Несколько примеров с валидации
n_show = 4
idxs = np.random.choice(len(x_val), n_show, replace=False)
sample_x = x_val[idxs] # uint8, модель сама поделит на 255
preds = best_model.predict(sample_x, batch_size=BATCH_SIZE)
pred_classes = np.argmax(preds, axis=-1)
true_classes = y_val[idxs]

fig, axes = plt.subplots(n_show, 3, figsize=(12, 3 * n_show))
for i in range(n_show):
    axes[i, 0].imshow(sample_x[i]); axes[i, 0].set_title('Изображение'); axes[i, 0].axis('off')
    axes[i, 1].imshow(true_classes[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1); axes[i, 1].set_title('Истинная маска'); axes[i, 1].axis('off')
    axes[i, 2].imshow(pred_classes[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1); axes[i, 2].set_title('Предсказание'); axes[i, 2].axis('off')
plt.tight_layout(); plt.show()

del best_model, preds, sample_x
tf.keras.backend.clear_session(); gc.collect()
```

Epoch 1/7
238/238 - 41s - 172ms/step - accuracy: 0.4356 - loss: 1.2616 - val_accuracy: 0.3729 - val_loss: 1.2821
Epoch 2/7
238/238 - 13s - 54ms/step - accuracy: 0.6848 - loss: 0.8573 - val_accuracy: 0.6002 - val_loss: 1.0168
Epoch 3/7
238/238 - 13s - 56ms/step - accuracy: 0.7303 - loss: 0.7553 - val_accuracy: 0.5937 - val_loss: 1.0197
Epoch 4/7
238/238 - 13s - 55ms/step - accuracy: 0.7465 - loss: 0.7100 - val_accuracy: 0.6538 - val_loss: 0.9545
Epoch 5/7
238/238 - 13s - 54ms/step - accuracy: 0.7645 - loss: 0.6702 - val_accuracy: 0.6585 - val_loss: 0.9139
Epoch 6/7
238/238 - 13s - 53ms/step - accuracy: 0.7731 - loss: 0.6472 - val_accuracy: 0.6745 - val_loss: 0.8818
Epoch 7/7
238/238 - 13s - 54ms/step - accuracy: 0.7840 - loss: 0.6219 - val_accuracy: 0.6700 - val_loss: 0.9020
1/1 ----- 1s 1s/step

Рисунок 18 – Код визуализации предсказаний лучшей модели

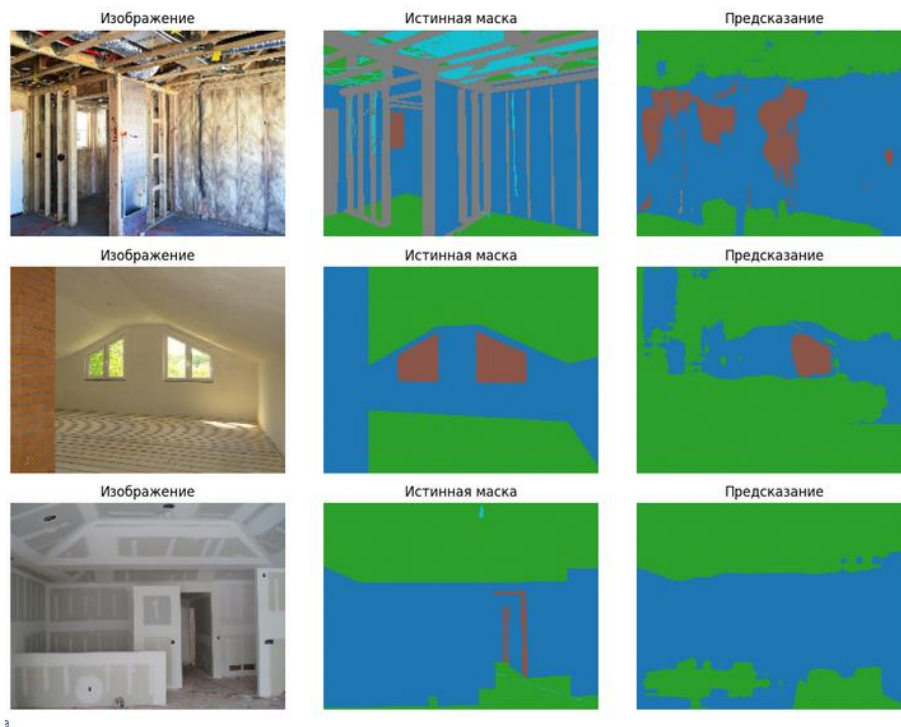


Рисунок 19 – Примеры предсказаний лучшей модели (Keras, 5 классов)

1.2. реализация с PyTorch: 5 классов, 10 экспериментов.

1.2.1. Подготовка данных (PyTorch).

Данные подготовлены для PyTorch: X хранится как uint8 NCHW (1900, 3, 192, 256), Y – как int64 (требование CrossEntropyLoss). Маппинг MAP_16_TO_5 аналогичен Keras-версии. Распределение классов идентично: фон – 47 798 941, стены – 32 926 001, проёмы – 9 025 507, крыша – 2 931 560, прочее – 706 791.

```

# 1. ПОДГОТОВКА ДАННЫХ: 16 -> 5 классов (PyTorch)
import numpy as np
import gc
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

NUM_CLASSES = 5

# Стандартная палитра 16 классов датасета construction_256x192.
CLASS_COLORS_16 = [
    (0,0,0),(128,0,0),(0,128,0),(128,128,0),
    (0,0,128),(128,0,128),(0,128,128),(128,128,128),
    (64,0,0),(192,0,0),(64,128,0),(192,128,0),
    (64,0,128),(192,0,128),(64,128,128),(192,128,128),
]

# 16 -> 5: 0=фон, 1=стены, 2=проёмы, 3=крыша, 4=прочее
MAP_16_TO_5 = np.array([
    0,
    1,1,1,
    2,2,2,2,
    3,3,3,3,
    4,4,4,4,
], dtype=np.int64)

def build_lut(colors):
    lut = np.full(256*3, 255, dtype=np.uint8)
    for idx,(r,g,b) in enumerate(colors):
        lut[(r<<16)|(g<<8)|b] = idx
    return lut

def seg_to_class5(seg, lut):
    s = np.asarray(seg, dtype=np.uint8)
    if s.ndim == 2:      s = np.stack([s]*3, axis=-1)
    if s.shape[-1] == 4: s = s[..., :3]
    keys = ((s[...,0].astype(np.uint32)<<16) |
            (s[...,1].astype(np.uint32)<<8) |
            s[...,2].astype(np.uint32))
    c16 = lut[keys]
    c16 = np.where(c16==255, 0, c16)
    return MAP_16_TO_5[c16].astype(np.int8)

# X (uint8)
print('Готовим X...')
first = np.asarray(train_images[0], dtype=np.uint8)
H, W = first.shape[:2]
print(f' Реальная форма картинки: H={H}, W={W}')

def stack_imgs(lst):
    out = np.empty((len(lst), H, W, 3), dtype=np.uint8)
    for i, im in enumerate(lst):
        a = np.asarray(im, dtype=np.uint8)
        if a.ndim == 2:      a = np.stack([a]*3, axis=-1)
        if a.shape[-1] == 4: a = a[..., :3]
        out[i] = a
    return out

x_train = stack_imgs(train_images)
x_val   = stack_imgs(val_images)
print(f' x_train {x_train.shape} ~{x_train.nbytes/1024/1024:.1f} MB')
del train_images, val_images
gc.collect()

```

Рисунок 20 – Подготовка данных: 16 → 5 классов (PyTorch, часть 1)


```

# Y (int8 карта классов)
print('Готовим Y...')
LUT = build_lut(CLASS_COLORS_16)

probe = np.asarray(train_segments[0], dtype=np.uint8)
if probe.shape[-1]==4: probe = probe[...,:3]
pk = ((probe[...,:0].astype(np.uint32)<<16)|(probe[...,:1].astype(np.uint32)<<8)|probe[...,:2].astype(np.uint32))
known = (LUT[pk]!=255).mean()
print(f' Известных пикселей на первой маске: {known:.3f}')
if known < 0.5:
    print(' >> Палитра не подошла, определяем 16 цветов по частоте...')
    ka=[]
    for seg in train_segments:
        a=np.asarray(seg,dtype=np.uint8)
        if a.shape[-1]==4: a=a[...,:3]
        ka.append((((a[...,:0].astype(np.uint32)<<16)|(a[...,:1].astype(np.uint32)<<8)|a[...,:2].astype(np.uint32))).ravel())
    ka=np.concatenate(ka); vals,cnt=np.unique(ka,return_counts=True)
    top=vals[np.argsort(-cnt)[:16]]
    LUT=np.full(256*3,255,dtype=np.uint8)
    for idx,k in enumerate(top): LUT[k]=idx
    del ka,vals,cnt,top; gc.collect()

y_train = np.empty((len(train_segments), H, W), dtype=np.int8)
for i,seg in enumerate(train_segments): y_train[i] = seg_to_class5(seg, LUT)
y_val = np.empty((len(val_segments), H, W), dtype=np.int8)
for i,seg in enumerate(val_segments): y_val[i] = seg_to_class5(seg, LUT)
print(f' y_train {y_train.shape} ~{y_train.nbytes/1024/1024:.1f} MB')

uniq,cnts = np.unique(y_train, return_counts=True)
print(' Классы в y_train:', dict(zip(uniq.tolist(), cnts.tolist())))

del train_segments, val_segments
gc.collect()

# torch-тензоры (NCHW) |
# X как uint8 NCHW; Y как long (CrossEntropy требует int64 индексы)
xt_train = torch.from_numpy(x_train).permute(0,3,1,2).contiguous() # uint8
xt_val = torch.from_numpy(x_val).permute(0,3,1,2).contiguous()
yt_train = torch.from_numpy(y_train.astype(np.int64))
yt_val = torch.from_numpy(y_val.astype(np.int64))
print('xt_train', xt_train.shape, xt_train.dtype, '| yt_train', yt_train.shape, yt_train.dtype)

del x_train, x_val, y_train, y_val
gc.collect()

```

```

Device: cuda
Готовим X...
    Реальная форма картинки: H=192, W=256
    x_train (1900, 192, 256, 3) ~267.2 MB
Готовим Y...
    Известных пикселей на первой маске: 0.000
    >> Палитра не подошла, определяем 16 цветов по частоте...
    y_train (1900, 192, 256) ~89.1 MB
    Классы в y_train: {0: 47798941, 1: 32926001, 2: 9025507, 3: 2931560, 4: 706791}
xt_train torch.Size([1900, 3, 192, 256]) torch.uint8 | yt_train torch.Size([1900, 192, 256]) torch.int64
0

```

Рисунок 21 – Формирование тензоров Y и конвертация в формат PyTorch

1.2.2. Модель U-Net (PyTorch).

U-Net реализована через nn.Module. Определён словарь АСТ с активациями relu, elu, selu, linear (nn.Identity). Блок DoubleConv – два Conv2D + BatchNorm + активация. Декодер использует ConvTranspose2d со stride=2. Нормализация 1/255 внутри forward. Паддинг до кратности 8 применяется перед подачей в сеть.

```

# 2. U-Net на PyTorch с настраиваемыми параметрами
import torch.nn as nn

ACT = {
    'relu': lambda: nn.ReLU(inplace=True),
    'elu': lambda: nn.ELU(inplace=True),
    'selu': lambda: nn.SELU(inplace=True),
    'linear': lambda: nn.Identity(),
}

class DoubleConv(nn.Module):
    def __init__(self, cin, cout, k, activation):
        super().__init__()
        p = k // 2
        self.net = nn.Sequential(
            nn.Conv2d(cin, cout, k, padding=p), nn.BatchNorm2d(cout), ACT[activation](),
            nn.Conv2d(cout, cout, k, padding=p), nn.BatchNorm2d(cout), ACT[activation]()
        )
    def forward(self, x): return self.net(x)

class UNet(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES, filters=16, kernel_size=3, activation='relu'):
        super().__init__()
        f = filters
        self.e1 = DoubleConv(3, f, kernel_size, activation)
        self.e2 = DoubleConv(f, f*2, kernel_size, activation)
        self.e3 = DoubleConv(f*2, f*4, kernel_size, activation)
        self.b = DoubleConv(f*4, f*8, kernel_size, activation)
        self.pool = nn.MaxPool2d(2)
        self.up3 = nn.ConvTranspose2d(f*8, f*4, 2, stride=2)
        self.d3 = DoubleConv(f*8, f*4, kernel_size, activation)
        self.up2 = nn.ConvTranspose2d(f*4, f*2, 2, stride=2)
        self.d2 = DoubleConv(f*4, f*2, kernel_size, activation)
        self.up1 = nn.ConvTranspose2d(f*2, f, 2, stride=2)
        self.d1 = DoubleConv(f*2, f, kernel_size, activation)
        self.out = nn.Conv2d(f, num_classes, 1)

    def forward(self, x):
        x = x.float() / 255.0 # нормализация внутри модели
        c1 = self.e1(x); p1 = self.pool(c1)
        c2 = self.e2(p1); p2 = self.pool(c2)
        c3 = self.e3(p2); p3 = self.pool(c3)
        b = self.b(p3)
        u3 = self.up3(b); u3 = torch.cat([u3, c3], dim=1); d3 = self.d3(u3)
        u2 = self.up2(d3); u2 = torch.cat([u2, c2], dim=1); d2 = self.d2(u2)
        u1 = self.up1(d2); u1 = torch.cat([u1, c1], dim=1); d1 = self.d1(u1)
        return self.out(d1) # логиты (B, C, H, W)

# Размеры кратны 8 (3 пулинга). Подгоним вход при необходимости.
_, _, IMG_H, IMG_W = xt_train.shape
print(f'Вход: ({IMG_H}, {IMG_W})')
PAD_H = (8 - IMG_H % 8) % 8
PAD_W = (8 - IMG_W % 8) % 8
print(f'Паддинг до кратности 8: +{PAD_H}h, +{PAD_W}w')

--- Вход: (192, 256)
Паддинг до кратности 8: +0h, +0w

```

Рисунок 22 – Реализация U-Net на PyTorch

1.2.3. Проведение 10 экспериментов (7 эпох).

EPOCHS=7, BATCH_SIZE=8. Функция потерь – F.cross_entropy, оптимизатор – Adam (lr=1e-3). Конфигурации экспериментов: те же комбинации filters (8, 16, 32), kernel_size (3, 5, 7) и активаций (relu, elu, selu, linear).

```

# 3. 10 ЭКСПЕРИМЕНТОВ (одинаковое число эпох = 7)
import torch.nn.functional as F
import time

EPOCHS = 7
BATCH_SIZE = 8

train_loader = DataLoader(TensorDataset(xt_train, yt_train),
                          batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(TensorDataset(xt_val, yt_val),
                        batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)

def pad_to8(t):
    if PAD_H or PAD_W:
        return F.pad(t, (0, PAD_W, 0, PAD_H)) # (left,right,top,bottom)
    return t

@torch.no_grad()
def evaluate(model):
    model.eval()
    tot_loss, correct, total = 0.0, 0, 0
    for xb, yb in val_loader:
        xb = pad_to8(xb.to(device, non_blocking=True))
        yb = pad_to8(yb.to(device, non_blocking=True))
        logits = model(xb)
        loss = F.cross_entropy(logits, yb)
        tot_loss += loss.item() * xb.size(0)
        pred = logits.argmax(1)
        correct += (pred == yb).sum().item()
        total += yb.numel()
    return tot_loss/len(val_loader.dataset), correct/total

def train_one(model):
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    hist = {'train_loss':[], 'train_acc':[], 'val_loss':[], 'val_acc':[]}
    for ep in range(EPOCHS):
        model.train()
        tl, c, t = 0.0, 0, 0
        for xb, yb in train_loader:
            xb = pad_to8(xb.to(device, non_blocking=True))
            yb = pad_to8(yb.to(device, non_blocking=True))
            opt.zero_grad()
            logits = model(xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()
            opt.step()
            tl += loss.item()*xb.size(0)
            c += (logits.argmax(1)==yb).sum().item()
            t += yb.numel()
        tr_loss, tr_acc = tl/len(train_loader.dataset), c/t
        v_loss, v_acc = evaluate(model)
        hist['train_loss'].append(tr_loss); hist['train_acc'].append(tr_acc)
        hist['val_loss'].append(v_loss); hist['val_acc'].append(v_acc)
        print(f' эпоха {ep+1}/{EPOCHS} train_loss={tr_loss:.4f} acc={tr_acc:.4f}'
              f' | val_loss={v_loss:.4f} acc={v_acc:.4f}')
    return hist

experiments = [
    {'name':'exp01','filters':16,'kernel_size':3,'activation':'relu'},
    {'name':'exp02','filters':32,'kernel_size':3,'activation':'relu'},
    {'name':'exp03','filters':8,'kernel_size':3,'activation':'relu'},
    {'name':'exp04','filters':16,'kernel_size':5,'activation':'relu'},
    {'name':'exp05','filters':16,'kernel_size':7,'activation':'relu'},
    {'name':'exp06','filters':16,'kernel_size':3,'activation':'elu'},
    {'name':'exp07','filters':16,'kernel_size':3,'activation':'selu'},
    {'name':'exp08','filters':16,'kernel_size':3,'activation':'linear'},
    {'name':'exp09','filters':32,'kernel_size':5,'activation':'elu'},
    {'name':'exp10','filters':8,'kernel_size':5,'activation':'selu'},
]

```

Рисунок 23 – Конфигурации экспериментов и функции обучения

```

results = []
for cfg in experiments:
    print('\n' + '='*70)
    print(f"{cfg['name']}: filters={cfg['filters']}, kernel_size={cfg['kernel_size']}, "
          f"activation={cfg['activation']}")
    print('='*70)
    t0 = time.time()

    model = UNet(NUM_CLASSES, cfg['filters'], cfg['kernel_size'], cfg['activation']).to(device)
    hist = train_one(model)

    results.append({
        'name':cfg['name'], 'filters':cfg['filters'],
        'kernel_size':cfg['kernel_size'], 'activation':cfg['activation'],
        'train_acc':hist['train_acc'], 'val_acc':hist['val_acc'],
        'train_loss':hist['train_loss'], 'val_loss':hist['val_loss'],
        'final_val_acc':hist['val_acc'][-1], 'final_val_loss':hist['val_loss'][-1],
    })
    print(f" время: {time.time()-t0:.1f}c")

    # Очистка ОЗУ/VRAM между экспериментами
    del model, hist
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

print('\nВсе эксперименты завершены.')

```

```

" эпоха 1/7 train_loss=1.0505 acc=0.6198 | val_loss=1.1268 acc=0.5619
  эпоха 2/7 train_loss=0.7926 acc=0.7149 | val_loss=0.9853 acc=0.6501
  эпоха 3/7 train_loss=0.7410 acc=0.7315 | val_loss=0.9248 acc=0.6765
  эпоха 4/7 train_loss=0.7077 acc=0.7440 | val_loss=1.0194 acc=0.6313
  эпоха 5/7 train_loss=0.6750 acc=0.7581 | val_loss=0.9818 acc=0.6461
  эпоха 6/7 train_loss=0.6573 acc=0.7660 | val_loss=0.9365 acc=0.6663
  эпоха 7/7 train_loss=0.6431 acc=0.7697 | val_loss=0.8939 acc=0.6796
  время: 105.1c

```

```

=====
exp07: filters=16, kernel_size=3, activation=selu
=====
  эпоха 1/7 train_loss=1.1691 acc=0.6015 | val_loss=1.2497 acc=0.5419
  эпоха 2/7 train_loss=0.8411 acc=0.7037 | val_loss=1.2162 acc=0.5066
  эпоха 3/7 train_loss=0.7703 acc=0.7210 | val_loss=1.0215 acc=0.6231
  эпоха 4/7 train_loss=0.7361 acc=0.7361 | val_loss=0.9901 acc=0.6231
  эпоха 5/7 train_loss=0.7059 acc=0.7499 | val_loss=0.9522 acc=0.6790
  эпоха 6/7 train_loss=0.6851 acc=0.7587 | val_loss=0.9380 acc=0.6572
  эпоха 7/7 train_loss=0.6758 acc=0.7630 | val_loss=0.9067 acc=0.6825
  время: 104.8c

```

```

=====
exp08: filters=16, kernel_size=3, activation=linear
=====
  эпоха 1/7 train_loss=1.1628 acc=0.6126 | val_loss=1.1431 acc=0.6070
  эпоха 2/7 train_loss=0.8631 acc=0.6912 | val_loss=1.1139 acc=0.5767
  эпоха 3/7 train_loss=0.8077 acc=0.7026 | val_loss=1.0010 acc=0.6278
  эпоха 4/7 train_loss=0.7871 acc=0.7116 | val_loss=0.9917 acc=0.6228
  эпоха 5/7 train_loss=0.7723 acc=0.7178 | val_loss=1.0099 acc=0.6377
  эпоха 6/7 train_loss=0.7581 acc=0.7238 | val_loss=1.0681 acc=0.6329
  эпоха 7/7 train_loss=0.7506 acc=0.7282 | val_loss=1.0252 acc=0.6359
  время: 99.9c

```

Рисунок 24 – Ход обучения и сохранение результатов

```

# 4. ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ
plt.figure(figsize=(14,6))
for r in results:
    plt.plot(range(1,EPOCHS+1), r['val_acc'], marker='o',
             label=f"{r['name']} f={r['filters']} k={r['kernel_size']} {r['activation']}")
plt.title('Точность на валидации по эпохам (10 экспериментов, PyTorch)')
plt.xlabel('Эпоха'); plt.ylabel('val_accuracy'); plt.grid(alpha=0.3)
plt.legend(bbox_to_anchor=(1.02,1), loc='upper left', fontsize=9)
plt.tight_layout(); plt.show()

plt.figure(figsize=(14,6))
for r in results:
    plt.plot(range(1,EPOCHS+1), r['val_loss'], marker='o',
             label=f"{r['name']} f={r['filters']} k={r['kernel_size']} {r['activation']}")
plt.title('Loss на валидации по эпохам (10 экспериментов, PyTorch)')
plt.xlabel('Эпоха'); plt.ylabel('val_loss'); plt.grid(alpha=0.3)
plt.legend(bbox_to_anchor=(1.02,1), loc='upper left', fontsize=9)
plt.tight_layout(); plt.show()

plt.figure(figsize=(12,6))
names=[r['name'] for r in results]; finals=[r['final_val_acc'] for r in results]
bars=plt.bar(names, finals, color='steelblue')
plt.title('Финальная val_accuracy по экспериментам')
plt.ylabel('val_accuracy'); plt.xticks(rotation=45)
for b,v in zip(bars,finals):
    plt.text(b.get_x()+b.get_width()/2, v+0.005, f'{v:.3f}', ha='center', fontsize=9)
plt.grid(axis='y', alpha=0.3); plt.tight_layout(); plt.show()

print('\n'+'*80)
print(f'{r["name"]<8}{r["filters"]<10}{r["kernel_size"]<10}{r["activation"]<14}{r["val_acc"]<12}{r["val_loss"]<12}')
print('-'*80)
for r in results:
    print(f"{r['name']<8}{r['filters']<10}{r['kernel_size']<10}"
          f"{r['activation']<14}{r['final_val_acc']<12.4f}{r['final_val_loss']<12.4f}")
print('-'*80)
best = max(results, key=lambda r: r['final_val_acc'])
print(f"Лучший: {best['name']} - filters={best['filters']}, "
      f"kernel_size={best['kernel_size']}, activation={best['activation']}, "
      f"val_acc={best['final_val_acc']:.4f}")

```

Рисунок 25 – Вывод обучения (эксперименты 6–8) и код визуализации

1.2.4. Визуализация результатов.

Наилучший результат в PyTorch-версии показал exp05 (filters=16, kernel_size=7, relu) – val_acc=0.712, val_loss=0.8299. Линейная активация (exp08) дала наихудший результат – val_acc=0.636.

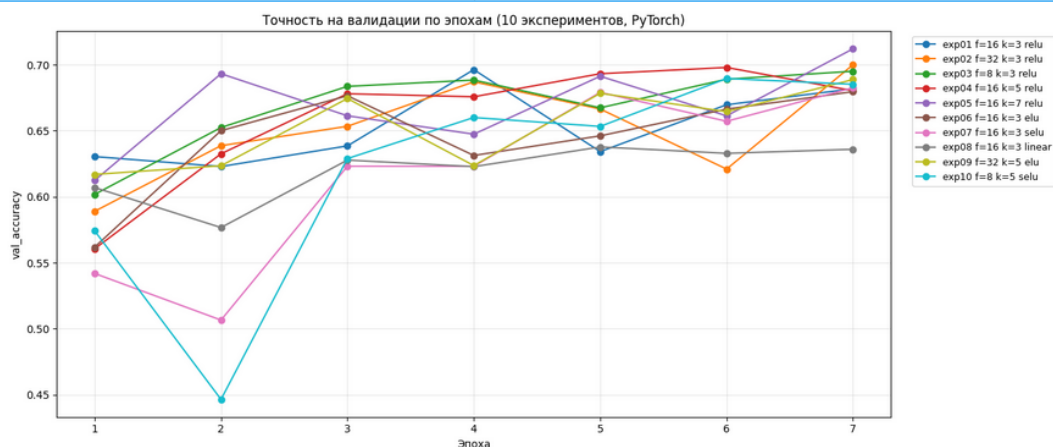


Рисунок 26 – Кривые val_ассурагу по эпохам (10 экспериментов, PyTorch)

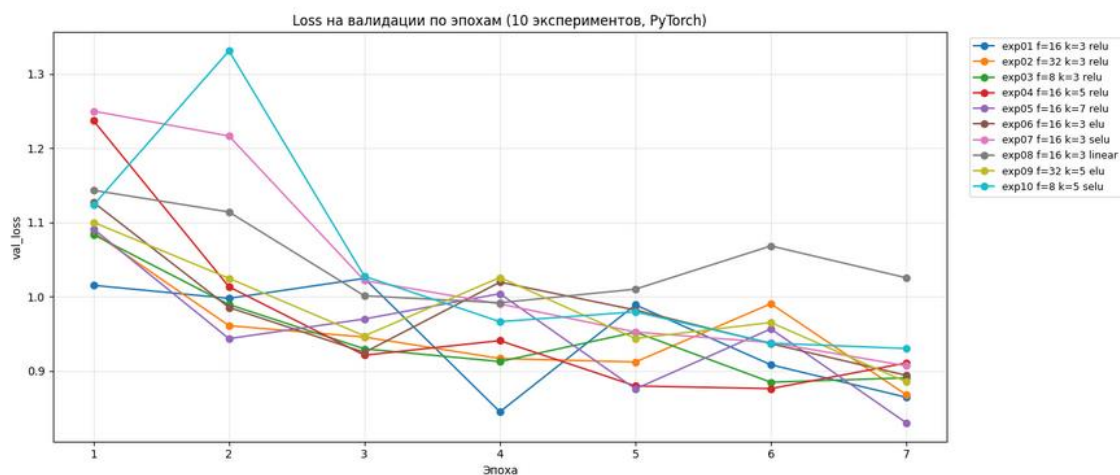


Рисунок 27 – Кривые val_loss по эпохам (10 экспериментов, PyTorch)

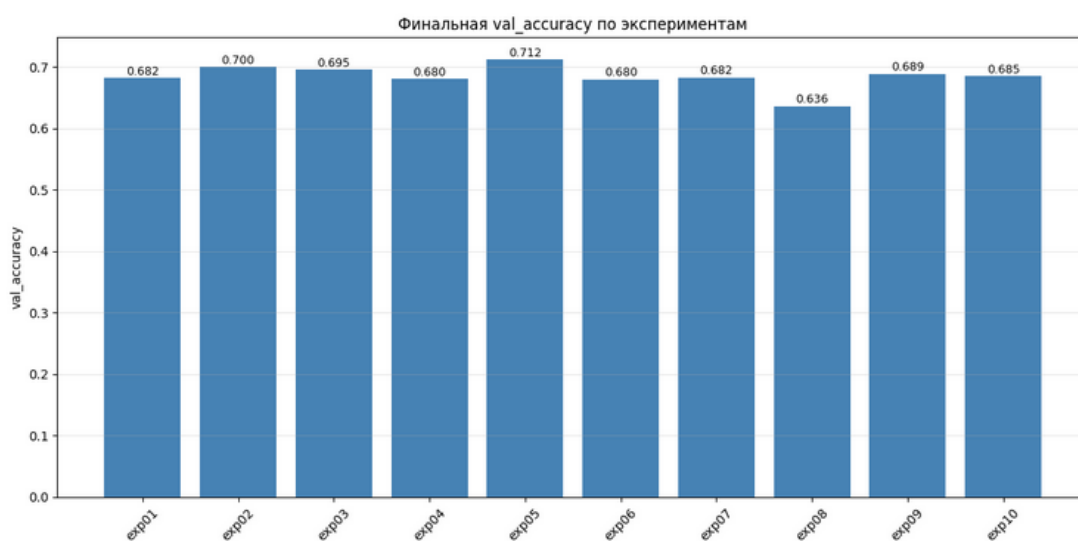


Рисунок 28 – Финальная val_accuracy по экспериментам (PyTorch)

name	filters	kernel	activation	val_acc	val_loss
exp01	16	3	relu	0.6818	0.8643
exp02	32	3	relu	0.6999	0.8676
exp03	8	3	relu	0.6950	0.8905
exp04	16	5	relu	0.6803	0.9105
exp05	16	7	relu	0.7120	0.8299
exp06	16	3	elu	0.6796	0.8939
exp07	16	3	selu	0.6825	0.9067
exp08	16	3	linear	0.6359	1.0252
exp09	32	5	elu	0.6887	0.8851
exp10	8	5	selu	0.6852	0.9300

Лучший: exp05 – filters=16, kernel_size=7, activation=relu, val_acc=0.7120

Рисунок 29 – Итоговая таблица экспериментов (PyTorch)

1.2.5. Предсказания лучшей модели.

Лучшая модель (exp05: filters=16, kernel_size=7, relu) переобучена и применена для предсказания масок на валидационной выборке. Финальная точность: val_acc=0.7028.


```

# 5. ВИЗУАЛИЗАЦИЯ ПРЕДСКАЗАНИЙ ЛУЧШЕЙ МОДЕЛИ
gc.collect()
if torch.cuda.is_available(): torch.cuda.empty_cache()

best_model = UNet(NUM_CLASSES, best['filters'], best['kernel_size'],
                  best['activation']).to(device)
_ = train_one(best_model)

best_model.eval()
n_show = 4
idxs = np.random.choice(len(xt_val), n_show, replace=False)
with torch.no_grad():
    xb = pad_to8(xt_val[idxs].to(device))
    logits = best_model(xb)
    pred = logits.argmax(1).cpu().numpy()[idxs, :IMG_H, :IMG_W]

ings = xt_val[idxs].permute(0,2,3,1).numpy() # (n,H,W,3) uint8
true = yt_val[idxs].numpy()

fig, axes = plt.subplots(n_show, 3, figsize=(13, 3.0*n_show))
for i in range(n_show):
    axes[i,0].imshow(ings[i]); axes[i,0].set_title('Изображение'); axes[i,0].axis('off')
    axes[i,1].imshow(true[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1)
    axes[i,1].set_title('Истинная маска'); axes[i,1].axis('off')
    axes[i,2].imshow(pred[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1)
    axes[i,2].set_title('Предсказание'); axes[i,2].axis('off')
plt.tight_layout(); plt.show()

del best_model, logits, xb
gc.collect()
if torch.cuda.is_available(): torch.cuda.empty_cache()

```

Рисунок 30 – Код визуализации предсказаний лучшей модели (PyTorch)

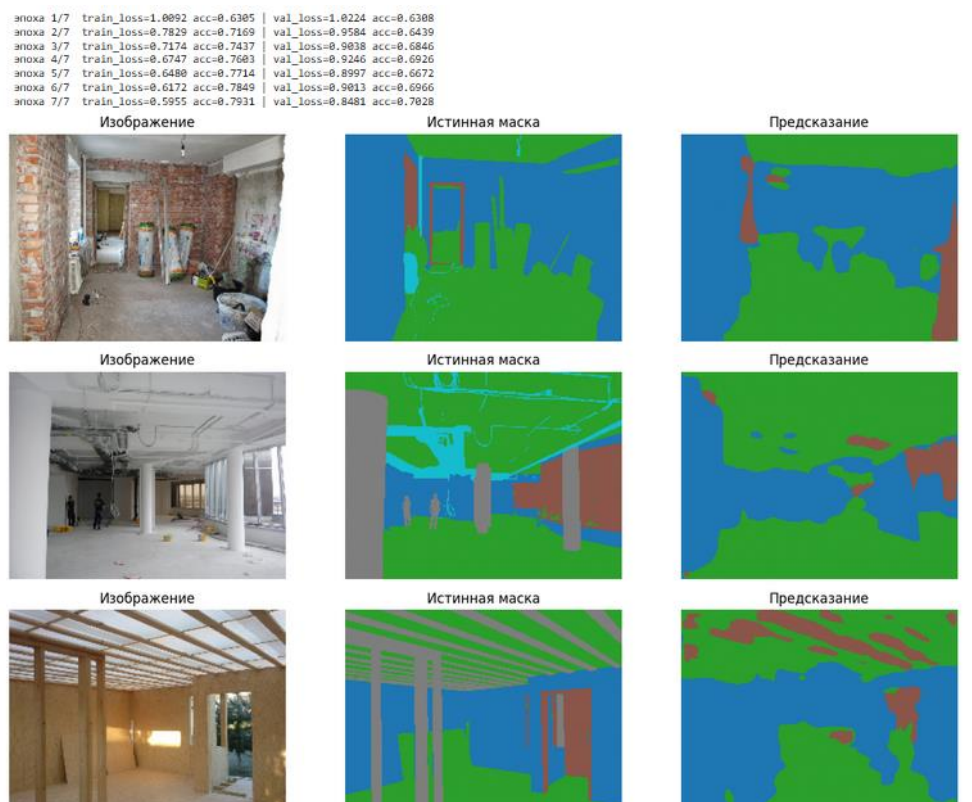


Рисунок 31 – Примеры предсказаний лучшей модели (PyTorch, 5 классов)

Задание Pro: 7 классов, каноническая U-Net, 100 эпох.

2.1. Отличие канонической U-Net от simpleUnet.

Принципиальное отличие канонической U-Net от simpleUnet заключается в следующем.

В simpleUnet декодер использует UpSampling2D – простое билинейное масштабирование без обучаемых параметров. Skip-соединения присутствуют не на всех уровнях, что ведёт к потере пространственной информации.

В канонической U-Net (Ronneberger et al., 2015): (1) декодер использует Conv2DTranspose – обучаемую транспонированную свёртку со stride=2; (2) skip-соединения (конкатенация feature-карт энкодера и декодера) применяются на КАЖДОМ уровне иерархии; (3) 4 уровня энкодера вместо 3; (4) Dropout-регуляризация на буттлнеке и последнем блоке энкодера; (5) инициализация весов he_normal. Схема фильтров: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ (буттлнек).

2.2. Загрузка данных.

Загрузка датасета, оригинальных изображений и масок аналогична заданию 1. 1900 обучающих и 100 проверочных пар.

```
# Импортируем модели keras: Model
from tensorflow.keras.models import Model

# Импортируем стандартные слои keras
from tensorflow.keras.layers import Input, Conv2DTranspose, concatenate, Activation
from tensorflow.keras.layers import MaxPooling2D, Conv2D, BatchNormalization, UpSampling2D

# Импортируем оптимизатор Adam
from tensorflow.keras.optimizers import Adam

# Импортируем модуль pyplot библиотеки matplotlib для построения графиков
import matplotlib.pyplot as plt

# Импортируем модуль image для работы с изображениями
from tensorflow.keras.preprocessing import image

# Импортируем библиотеку numpy
import numpy as np

# Импортируем метод деления выборки
from sklearn.model_selection import train_test_split

# загрузка файлов по HTML ссылке
import gdown

# Для работы с файлами
import os

# Для генерации случайных чисел
import random

import time

# импортируем модель Image для работы с изображениями
from PIL import Image

# очистка ОЗУ
import gc
```

Рисунок 32 – Импорт библиотек (Pro-вариант)

```
# Загрузка датасета из облака

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/114/construction_256x192.zip', None, quiet=False)
#gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/114/construction_512x384.zip', None, quiet=False)

!unzip -q 'construction_256x192.zip' # распаковываем архив

Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/114/construction_256x192.zip
To: /content/construction_256x192.zip
100%|██████████| 214M/214M [00:09<00:00, 21.7MB/s]
```

Предварительная подготовка данных

```
# Глобальные параметры

IMG_WIDTH = 192          # Ширина картинки
IMG_HEIGHT = 256         # Высота картинки
NUM_CLASSES = 16         # Задаем количество классов на изображении
TRAIN_DIRECTORY = 'train' # Название папки с файлами обучающей выборки
VAL_DIRECTORY = 'val'     # Название папки с файлами проверочной выборки

train_images = [] # Создаем пустой список для хранения оригинальных изображений обучающей выборки
val_images = []   # Создаем пустой список для хранения оригинальных изображений проверочной выборки

cur_time = time.time() # Засекаем текущее время

# Проходим по всем файлам в каталоге по указанному пути
for filename in sorted(os.listdir(TRAIN_DIRECTORY+'original')):
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    train_images.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'original',filename),
                                             target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок обучающей выборки
print ('Обучающая выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в обучающей выборке
print ('Количество изображений: ', len(train_images))

cur_time = time.time() # Засекаем текущее время

# Проходим по всем файлам в каталоге по указанному пути
for filename in sorted(os.listdir(VAL_DIRECTORY+'original')):
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    val_images.append(image.load_img(os.path.join(VAL_DIRECTORY+'original',filename),
                                             target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок проверочной выборки
print ('Проверочная выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в проверочной выборке
print ('Количество изображений: ', len(val_images))

Обучающая выборка загружена. Время загрузки: 0.3с
Количество изображений: 1900
Проверочная выборка загружена. Время загрузки: 0.01с
Количество изображений: 100
```

Рисунок 34 – Загрузка изображений

2.3. Подготовка данных: 16 → 7 классов.

Задан маппинг MAP_16_TO_7 на 7 семантических категорий. Распределение классов в y_train: FLOOR – 47 798 941 (51.18%), CEILING – 25 679 475 (27.50%), WALL – 13 411 319 (14.36%), APERTURE – 3 783 583 (4.05%), SUPPORT – 1 483 145 (1.59%), INVENTORY – 879 546 (0.94%), OTHER – 352 871 (0.38%). Объём: X – 281.2 МБ, Y – 93.8 МБ.

```

# ШАГ 2. ПОДГОТОВКА ДАННЫХ: 16 -> 7 классов
# =====
import numpy as np
import gc
import tensorflow as tf

# 7 целевых классов:
# 0 = FLOOR
# 1 = CEILING
# 2 = WALL
# 3 = APERTURE / DOOR / WINDOW
# 4 = COLUMN / RAILINGS / LADDER
# 5 = INVENTORY
# 6 = LAMP / WIRE / BEAM / EXTERNAL / BATTERY / PEOPLE
NUM_CLASSES = 7

CLASS_NAMES = ['FLOOR', 'CEILING', 'WALL', 'APERTURE', 'SUPPORT', 'INVENTORY', 'OTHER']

# --- Стандартная палитра 16 классов датасета construction_256x192 ---
# Если фактические цвета в масках другие, ниже сработает авто-детектор.
CLASS_COLORS_16 = [
    (0, 0, 0), # 0
    (128, 0, 0), # 1
    (0, 128, 0), # 2
    (128, 128, 0), # 3
    (0, 0, 128), # 4
    (128, 0, 128), # 5
    (0, 128, 128), # 6
    (128, 128, 128), # 7
    (64, 0, 0), # 8
    (192, 0, 0), # 9
    (64, 128, 0), # 10
    (192, 128, 0), # 11
    (64, 0, 128), # 12
    (192, 0, 128), # 13
    (64, 128, 128), # 14
    (192, 128, 128), # 15
]

# --- Маппинг 16 -> 7 (по семантике из задания) ---
# Если фактический порядок 16 классов в маске не совпадает с этим распределением,
# можно поправить здесь. По умолчанию равномерно разносим 16 индексов по 7 группам:
# 0 -> FLOOR
# 1-2 -> CEILING
# 3-5 -> WALL
# 6-8 -> APERTURE
# 9-10 -> SUPPORT
# 11-12 -> INVENTORY
# 13-15 -> OTHER
MAP_16_TO_7 = np.array([
    0, # 0
    1, 1, # 1-2
    2, 2, 2, # 3-5
    3, 3, 3, # 6-8
    4, 4, # 9-10
    5, 5, # 11-12
    6, 6, 6, # 13-15
], dtype=np.int8)

def build_color_lut(colors):
    """RGB-цвет (24 бита) -> класс 0..15. 255 = неизвестный."""
    lut = np.full(256 * 3, 255, dtype=np.uint8)
    for idx, (r, g, b) in enumerate(colors):
        key = (r << 16) | (g << 8) | b
        lut[key] = idx
    return lut

```

Рисунок 36 – Маппинг 16 → 7 классов и функция build_color_lut

```

def seg_rgb_to_class7(seg_img_rgb, lut):
    """RGB-маска -> карта 7 классов (H, W) int8 через LUT."""
    seg = np.asarray(seg_img_rgb, dtype=np.uint8)
    if seg.ndim == 2:
        seg = np.stack([seg]*3, axis=-1)
    if seg.shape[-1] == 4:
        seg = seg[..., :3]
    r = seg[..., 0].astype(np.uint32)
    g = seg[..., 1].astype(np.uint32)
    b = seg[..., 2].astype(np.uint32)
    keys = (r << 16) | (g << 8) | b
    cls16 = lut[keys]
    cls16 = np.where(cls16 == 255, 0, cls16)
    return MAP_16_TO_7[cls16]

# ----- X -----
print('Готовим X (uint8)...')
first = np.asarray(train_images[0], dtype=np.uint8)
H, W = first.shape[:2]
print(f' Реальная форма картинки: H={H}, W={W}')

x_train = np.empty((len(train_images), H, W, 3), dtype=np.uint8)
for i, img in enumerate(train_images):
    a = np.asarray(img, dtype=np.uint8)
    if a.ndim == 2:
        a = np.stack([a]*3, axis=-1)
    if a.shape[-1] == 4:
        a = a[..., :3]
    x_train[i] = a

x_val = np.empty((len(val_images), H, W, 3), dtype=np.uint8)
for i, img in enumerate(val_images):
    a = np.asarray(img, dtype=np.uint8)
    if a.ndim == 2:
        a = np.stack([a]*3, axis=-1)
    if a.shape[-1] == 4:
        a = a[..., :3]
    x_val[i] = a

print(f' x_train: {x_train.shape} {x_train.dtype} ~{x_train.nbytes/1024/1024:.1f} MB')
print(f' x_val: {x_val.shape} {x_val.dtype} ~{x_val.nbytes/1024/1024:.1f} MB')

del train_images, val_images
gc.collect()

# ----- Y -----
print('\nГотовим Y (int8 карта 7 классов)...')
LUT = build_color_lut(CLASS_COLORS_16)

# Авто-проверка палитры на первой маске
probe = np.asarray(train_segments[0], dtype=np.uint8)
if probe.shape[-1] == 4: probe = probe[..., :3]
pk = ((probe[...,0].astype(np.uint32) << 16) |
      (probe[...,1].astype(np.uint32) << 8) |
      probe[...,2].astype(np.uint32))
known = (LUT[pk] != 255).mean()
print(f' Известных пикселей на первой маске: {known:.3f}')

if known < 0.5:
    print(' >> Стандартная палитра не подошла, определяем 16 самых частых цветов автоматически...')
    keys_all = []
    for seg in train_segments:
        a = np.asarray(seg, dtype=np.uint8)
        if a.shape[-1] == 4: a = a[..., :3]
        k = ((a[...,0].astype(np.uint32) << 16) |
              (a[...,1].astype(np.uint32) << 8) |
              a[...,2].astype(np.uint32))
        keys_all.append(k.ravel())
    keys_all = np.concatenate(keys_all)
    vals, counts = np.unique(keys_all, return_counts=True)
    top = vals[np.argsort(-counts)[:16]]
    print(f' Найдено уникальных цветов: {len(vals)}; взяли топ-16.')
    LUT = np.full(256 * 3, 255, dtype=np.uint8)

```

Рисунок 37 – Функция seg_rgb_to_class7 и формирование тензора X

```

LUT = np.full(256 * 3, 255, dtype=np.uint8)
for idx, k in enumerate(top):
    LUT[k] = idx
del keys_all, vals, counts, top
gc.collect()

y_train = np.empty((len(train_segments), H, W), dtype=np.int8)
for i, seg in enumerate(train_segments):
    y_train[i] = seg_rgb_to_class7(seg, LUT)

y_val = np.empty((len(val_segments), H, W), dtype=np.int8)
for i, seg in enumerate(val_segments):
    y_val[i] = seg_rgb_to_class7(seg, LUT)

print(f' y_train: {y_train.shape} {y_train.dtype} ~{y_train.nbytes/1024/1024:.1f} MB')
print(f' y_val: {y_val.shape} {y_val.dtype} ~{y_val.nbytes/1024/1024:.1f} MB')

# Распределение классов
uniq, cnts = np.unique(y_train, return_counts=True)
total = cnts.sum()
print('\n Распределение классов в y_train:')
for u, c in zip(uniq, cnts):
    name = CLASS_NAMES[u] if 0 <= u < len(CLASS_NAMES) else '?'
    print(f' {u} {name:<10s} {c:>10d} ({100.0*c/total:5.2f}%)')

del train_segments, val_segments
gc.collect()

print(f'\nИтог по памяти:')
print(f' X: {(x_train.nbytes + x_val.nbytes)/1024/1024:.1f} MB')
print(f' Y: {(y_train.nbytes + y_val.nbytes)/1024/1024:.1f} MB')

```

```

--- Готовим X (uint8)...
    Реальная форма картинки: H=192, W=256
    x_train: (1900, 192, 256, 3) uint8 ~267.2 MB
    x_val: (100, 192, 256, 3) uint8 ~14.1 MB

Готовим Y (int8 карта 7 классов)...
    Известных пикселей на первой маске: 0.000
    >> Стандартная палитра не подошла, определим 16 самых частых цветов автоматически...
    Найдено уникальных цветов: 27; взяли top-16.
    y_train: (1900, 192, 256) int8 ~89.1 MB
    y_val: (100, 192, 256) int8 ~4.7 MB

Распределение классов в y_train:
    0 FLOOR      47798941 (51.18%)
    1 CEILING    25679475 (27.50%)
    2 WALL       13411319 (14.36%)
    3 APERTURE    3783503 ( 4.05%)
    4 SUPPORT     1483145 ( 1.59%)
    5 INVENTORY   879546 ( 0.94%)
    6 OTHER       352871 ( 0.38%)

Итог по памяти:
    X: 281.2 MB
    Y: 93.8 MB

```

Рисунок 38 – Формирование тензора Y и распределение классов

2.4. Каноническая U-Net.

Реализован блок `conv_block` (двойная свёртка с BatchNorm, активацией и опциональным Dropout). Функция `build_unet` строит 4-уровневую сеть со skip-соединениями на каждом уровне декодера. `Base_filters=32` (схема: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$). Выходной слой – `Conv2D(7, 1, activation="softmax")`.

```

# =====
# ШАГ 3. КАНОНИЧЕСКИЙ U-Net (4 уровня, двойные свёртки,
#         Conv2DTranspose, skip-connections на каждом уровне)
# =====
from tensorflow.keras.layers import Rescaling, Dropout

def conv_block(x, filters, kernel_size=3, activation='relu', dropout=0.0):
    """Двойная свёртка как в каноническом U-Net: (Conv -> BN -> Act) x 2."""
    x = Conv2D(filters, kernel_size, padding='same', kernel_initializer='he_normal')(x)
    x = BatchNormalization()(x)
    x = Activation(activation)(x)
    x = Conv2D(filters, kernel_size, padding='same', kernel_initializer='he_normal')(x)
    x = BatchNormalization()(x)
    x = Activation(activation)(x)
    if dropout > 0:
        x = Dropout(dropout)(x)
    return x

def build_unet(img_h, img_w, num_classes=NUM_CLASSES,
               base_filters=32, kernel_size=3, activation='relu'):
    """Канонический 4-уровневый U-Net.

    Энкодер: base -> 2*base -> 4*base -> 8*base
    Bottleneck: 16*base
    Декодер: 8*base -> 4*base -> 2*base -> base (с concat skip-connections)
    """
    inp = Input(shape=(img_h, img_w, 3), dtype='uint8')
    x = Rescaling(1.0 / 255.0)(inp)

    # Encoder
    c1 = conv_block(x, base_filters, kernel_size, activation)
    p1 = MaxPooling2D()(c1)

    c2 = conv_block(p1, base_filters*2, kernel_size, activation)
    p2 = MaxPooling2D()(c2)

    c3 = conv_block(p2, base_filters*4, kernel_size, activation)
    p3 = MaxPooling2D()(c3)

    c4 = conv_block(p3, base_filters*8, kernel_size, activation, dropout=0.2)
    p4 = MaxPooling2D()(c4)

    # Bottleneck
    b = conv_block(p4, base_filters*16, kernel_size, activation, dropout=0.3)

    # Decoder – skip-connections на КАЖДОМ уровне
    u4 = Conv2DTranspose(base_filters*8, 2, strides=2, padding='same')(b)
    u4 = concatenate([u4, c4])
    d4 = conv_block(u4, base_filters*8, kernel_size, activation)

    u3 = Conv2DTranspose(base_filters*4, 2, strides=2, padding='same')(d4)
    u3 = concatenate([u3, c3])
    d3 = conv_block(u3, base_filters*4, kernel_size, activation)

    u2 = Conv2DTranspose(base_filters*2, 2, strides=2, padding='same')(d3)
    u2 = concatenate([u2, c2])
    d2 = conv_block(u2, base_filters*2, kernel_size, activation)

    u1 = Conv2DTranspose(base_filters, 2, strides=2, padding='same')(d2)
    u1 = concatenate([u1, c1])
    d1 = conv_block(u1, base_filters, kernel_size, activation)

    out = Conv2D(num_classes, 1, activation='softmax')(d1)

    model = Model(inputs=inp, outputs=out, name='unet_canonical')
    model.compile(
        optimizer=Adam(learning_rate=1e-3),
        loss='sparse_categorical_crossentropy'.

```

Рисунок 39 – Реализация conv_block и build_unet (каноническая U-Net)

```

model = Model(inputs=inp, outputs=out, name='unet_canonical')
model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
return model

# Размеры берём из реальной формы массивов
_, IMG_H, IMG_W, _ = x_train.shape
print(f'Размер входа: ({IMG_H}, {IMG_W}, 3)')

unet = build_unet(IMG_H, IMG_W, base_filters=32)
unet.summary()

```

Рисунок 40 – Компиляция модели и вызов build_unet(base_filters=32)

Размер входа: (192, 256, 3)
Model: "unet_canonical"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 192, 256, 3)	0	-
rescaling (Rescaling)	(None, 192, 256, 3)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 192, 256, 32)	896	rescaling[0][0]
batch_normalization (BatchNormalizatio..)	(None, 192, 256, 32)	128	conv2d[0][0]
activation (Activation)	(None, 192, 256, 32)	0	batch_normalizat..
conv2d_1 (Conv2D)	(None, 192, 256, 32)	9,248	activation[0][0]
batch_normalizatio.. (BatchNormalizatio..)	(None, 192, 256, 32)	128	conv2d_1[0][0]
activation_1 (Activation)	(None, 192, 256, 32)	0	batch_normalizat..
max_pooling2d (MaxPooling2D)	(None, 96, 128, 32)	0	activation_1[0][..
conv2d_2 (Conv2D)	(None, 96, 128, 64)	18,496	max_pooling2d[0][..
batch_normalizatio.. (BatchNormalizatio..)	(None, 96, 128, 64)	256	conv2d_2[0][0]
activation_2 (Activation)	(None, 96, 128, 64)	0	batch_normalizat..
conv2d_3 (Conv2D)	(None, 96, 128, 64)	36,928	activation_2[0][..
batch_normalizatio.. (BatchNormalizatio..)	(None, 96, 128, 64)	256	conv2d_3[0][0]
activation_3 (Activation)	(None, 96, 128, 64)	0	batch_normalizat..
max_pooling2d_1 (MaxPooling2D)	(None, 48, 64, 64)	0	activation_3[0][..
conv2d_4 (Conv2D)	(None, 48, 64, 128)	73,856	max_pooling2d_1[..
batch_normalizatio.. (BatchNormalizatio..)	(None, 48, 64, 128)	512	conv2d_4[0][0]
activation_4 (Activation)	(None, 48, 64, 128)	0	batch_normalizat..

Рисунок 41 – Краткая сводка архитектуры модели unet_canonical

2.5. Обучение на 100 эпохах.

EPOCHS=100, BATCH_SIZE=8. Коллбэки: EarlyStopping (patience=15, restore_best_weights=True), ReduceLROnPlateau (factor=0.5, patience=5, min_lr=1e-6), ModelCheckpoint (best_unet.weights.h5). Обучение остановилось на эпохе 42 (EarlyStopping), лучшие веса – с эпохи 27. Финальные метрики: train_acc=0.9390, val_acc=0.7441, лучшая val_acc=0.7475, лучший val_loss=0.8355.

```
#####
# ШАГ 4. ОБУЧЕНИЕ U-Net НА 100 ЭПОХАХ
#####
import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

EPOCHS = 100
BATCH_SIZE = 8

# tf.data датасеты – без копирования больших тензоров
def make_ds(x, y, batch, shuffle):
    ds = tf.data.Dataset.from_tensor_slices((x, y))
    if shuffle:
        ds = ds.shuffle(buffer_size=min(512, len(x)), reshuffle_each_iteration=True)
    return ds.batch(batch).prefetch(tf.data.AUTOTUNE)

train_ds = make_ds(x_train, y_train, BATCH_SIZE, shuffle=True)
val_ds = make_ds(x_val, y_val, BATCH_SIZE, shuffle=False)

# Коллбэки:
# - EarlyStopping: остановка при отсутствии улучшения val_loss 15 эпох
# - ReduceLROnPlateau: уменьшение шага при плато
# - ModelCheckpoint: сохраняем лучшие веса (только веса -> компактно)
callbacks = [
    EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, min_lr=1e-6, verbose=1),
    ModelCheckpoint('best_unet.weights.h5', monitor='val_loss',
                    save_best_only=True, save_weights_only=True, verbose=0),
]

# Свежая сессия перед длинным обучением
tf.keras.backend.clear_session(); gc.collect()
unet = build_unet(IMG_H, IMG_W, base_filters=32)

history = unet.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=2,
)

print('\nОбучение завершено.')
print(f"Финальная train_acc: {history.history['accuracy'][-1]:.4f}")
print(f"Финальная val_acc: {history.history['val_accuracy'][-1]:.4f}")
print(f"Лучшая val_acc: {max(history.history['val_accuracy']):.4f}")
print(f"Финальная val_loss: {history.history['val_loss'][-1]:.4f}")
print(f"Лучшая val_loss: {min(history.history['val_loss']):.4f}")
```

Рисунок 42 – Код обучения: датасеты, коллбэки, model.fit

```

Epoch 24: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
238/238 - 34s - 141ms/step - accuracy: 0.8805 - loss: 0.3483 - val_accuracy: 0.7316 - val_loss: 0.8963 - learning_rate: 0.0010
Epoch 25/100
238/238 - 33s - 140ms/step - accuracy: 0.8961 - loss: 0.3046 - val_accuracy: 0.7259 - val_loss: 0.9259 - learning_rate: 5.0000e-04
Epoch 26/100
238/238 - 34s - 142ms/step - accuracy: 0.9022 - loss: 0.2851 - val_accuracy: 0.7397 - val_loss: 0.8753 - learning_rate: 5.0000e-04
Epoch 27/100
238/238 - 34s - 143ms/step - accuracy: 0.9036 - loss: 0.2803 - val_accuracy: 0.7406 - val_loss: 0.8355 - learning_rate: 5.0000e-04
Epoch 28/100
238/238 - 34s - 141ms/step - accuracy: 0.9082 - loss: 0.2692 - val_accuracy: 0.7285 - val_loss: 0.8698 - learning_rate: 5.0000e-04
Epoch 29/100
238/238 - 33s - 140ms/step - accuracy: 0.9092 - loss: 0.2645 - val_accuracy: 0.7350 - val_loss: 0.8984 - learning_rate: 5.0000e-04
Epoch 30/100
238/238 - 33s - 140ms/step - accuracy: 0.9106 - loss: 0.2610 - val_accuracy: 0.7242 - val_loss: 0.9396 - learning_rate: 5.0000e-04
Epoch 31/100
238/238 - 33s - 139ms/step - accuracy: 0.9168 - loss: 0.2434 - val_accuracy: 0.7390 - val_loss: 0.8825 - learning_rate: 5.0000e-04
Epoch 32/100

Epoch 32: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
238/238 - 33s - 140ms/step - accuracy: 0.9196 - loss: 0.2342 - val_accuracy: 0.7385 - val_loss: 0.9303 - learning_rate: 5.0000e-04
Epoch 33/100
238/238 - 33s - 140ms/step - accuracy: 0.9256 - loss: 0.2173 - val_accuracy: 0.7401 - val_loss: 0.9053 - learning_rate: 2.5000e-04
Epoch 34/100
238/238 - 33s - 140ms/step - accuracy: 0.9288 - loss: 0.2080 - val_accuracy: 0.7461 - val_loss: 0.8789 - learning_rate: 2.5000e-04
Epoch 35/100
238/238 - 34s - 141ms/step - accuracy: 0.9296 - loss: 0.2053 - val_accuracy: 0.7426 - val_loss: 0.9399 - learning_rate: 2.5000e-04
Epoch 36/100
238/238 - 34s - 141ms/step - accuracy: 0.9310 - loss: 0.2006 - val_accuracy: 0.7405 - val_loss: 0.9190 - learning_rate: 2.5000e-04
Epoch 37/100

Epoch 37: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
238/238 - 34s - 141ms/step - accuracy: 0.9318 - loss: 0.1988 - val_accuracy: 0.7367 - val_loss: 0.9707 - learning_rate: 2.5000e-04
Epoch 38/100
238/238 - 33s - 141ms/step - accuracy: 0.9349 - loss: 0.1904 - val_accuracy: 0.7475 - val_loss: 0.9154 - learning_rate: 1.2500e-04
Epoch 39/100
238/238 - 33s - 141ms/step - accuracy: 0.9366 - loss: 0.1853 - val_accuracy: 0.7445 - val_loss: 0.9234 - learning_rate: 1.2500e-04
Epoch 40/100
238/238 - 33s - 140ms/step - accuracy: 0.9369 - loss: 0.1842 - val_accuracy: 0.7443 - val_loss: 0.9640 - learning_rate: 1.2500e-04
Epoch 41/100
238/238 - 33s - 139ms/step - accuracy: 0.9381 - loss: 0.1807 - val_accuracy: 0.7464 - val_loss: 0.9570 - learning_rate: 1.2500e-04
Epoch 42/100

Epoch 42: ReduceLROnPlateau reducing learning rate to 6.250000029685907e-05.
238/238 - 33s - 140ms/step - accuracy: 0.9390 - loss: 0.1774 - val_accuracy: 0.7441 - val_loss: 0.9586 - learning_rate: 1.2500e-04
Epoch 42: early stopping
Restoring model weights from the end of the best epoch: 27.

Обучение завершено.
Финальная train_acc: 0.9390
Финальная val_acc: 0.7441
Лучшая val_acc: 0.7475
Финальная val_loss: 0.9586
Лучшая val_loss: 0.8355

```

Рисунок 43 – Ход обучения (эпохи 24–42) и итоговые метрики

2.6. Графики обучения.

Кривая ассигуры на обучающей выборке стабильно растёт до ~ 0.94 , на валидационной – стабилизируется около $0.74\text{--}0.75$. Кривая loss на обучающей снижается до ~ 0.19 , на валидационной – колебалась около $0.9\text{--}1.0$. Переобучение ограничено коллбэками.

```

# =====
# ШАГ 5. ГРАФИКИ ОБУЧЕНИЯ
# =====
epochs_done = range(1, len(history.history['accuracy']) + 1)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(epochs_done, history.history['accuracy'], label='train', marker='.')
axes[0].plot(epochs_done, history.history['val_accuracy'], label='validation', marker='.')
axes[0].set_title('Accuracy'); axes[0].set_xlabel('Эпоха'); axes[0].set_ylabel('accuracy')
axes[0].grid(alpha=0.3); axes[0].legend()

axes[1].plot(epochs_done, history.history['loss'], label='train', marker='.')
axes[1].plot(epochs_done, history.history['val_loss'], label='validation', marker='.')
axes[1].set_title('Loss'); axes[1].set_xlabel('Эпоха'); axes[1].set_ylabel('loss')
axes[1].grid(alpha=0.3); axes[1].legend()

plt.tight_layout(); plt.show()

```

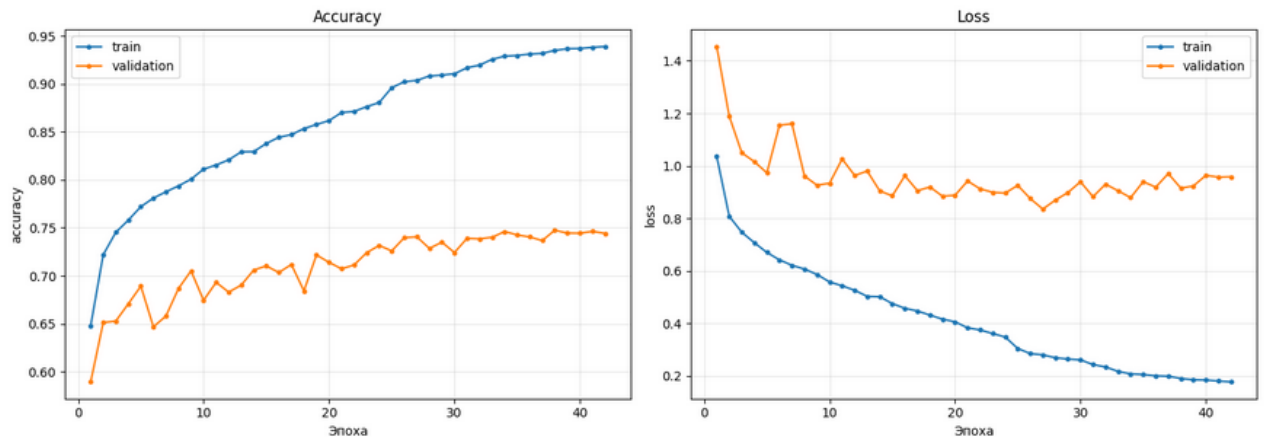


Рисунок 44 – Кривые Accuracy и Loss по эпохам (100 эпох, EarlyStopping на 42)

2.7. Визуализация предсказаний и метрика IoU.

Помимо accuracy вычислена метрика IoU (Intersection over Union) по каждому из 7 классов на полной валидационной выборке. Результаты IoU: FLOOR – 0.7886, CEILING – 0.6811, WALL – 0.3326, APERTURE – 0.1324, SUPPORT – 0.2122, INVENTORY – 0.0119, OTHER – 0.0630. Mean IoU – 0.3203.

```

# =====
# ШАГ 6. ВИЗУАЛИЗАЦИЯ ПРЕДСКАЗАНИЙ + IoU ПО КЛАССАМ
# =====

# --- Несколько примеров с валидации ---
n_show = 5
idxs = np.random.choice(len(x_val), n_show, replace=False)
sample_x = x_val[idxs]
preds = unet.predict(sample_x, batch_size=BATCH_SIZE, verbose=0)
pred_classes = np.argmax(preds, axis=-1)
true_classes = y_val[idxs]

fig, axes = plt.subplots(n_show, 3, figsize=(13, 3.0 * n_show))
for i in range(n_show):
    axes[i, 0].imshow(sample_x[i]); axes[i, 0].set_title('Изображение'); axes[i, 0].axis('off')
    axes[i, 1].imshow(true_classes[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1)
    axes[i, 1].set_title('Истинная маска'); axes[i, 1].axis('off')
    axes[i, 2].imshow(pred_classes[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1)
    axes[i, 2].set_title('Предсказание'); axes[i, 2].axis('off')
plt.tight_layout(); plt.show()

# --- IoU по каждому классу на полной валидации ---
print('\nIoU по каждому классу на валидации:')
batch = 16
val_pred_all = np.empty_like(y_val)
for s in range(0, len(x_val), batch):
    p = unet.predict(x_val[s:s+batch], verbose=0)
    val_pred_all[s:s+batch] = np.argmax(p, axis=-1).astype(np.int8)

ious = []
for c in range(NUM_CLASSES):
    inter = np.logical_and(y_val == c, val_pred_all == c).sum()
    union = np.logical_or(y_val == c, val_pred_all == c).sum()
    iou = inter / union if union > 0 else float('nan')
    ious.append(iou)
    print(f' {c} {CLASS_NAMES[c]:<10s} IoU = {iou:.4f}')

valid_ious = [v for v in ious if not np.isnan(v)]
print(f'\nMean IoU (по присутствующим классам): {np.mean(valid_ious):.4f}')

# Очистка
del preds, sample_x, val_pred_all
gc.collect()

```

Рисунок 45 – Код визуализации предсказаний и вычисления IoU

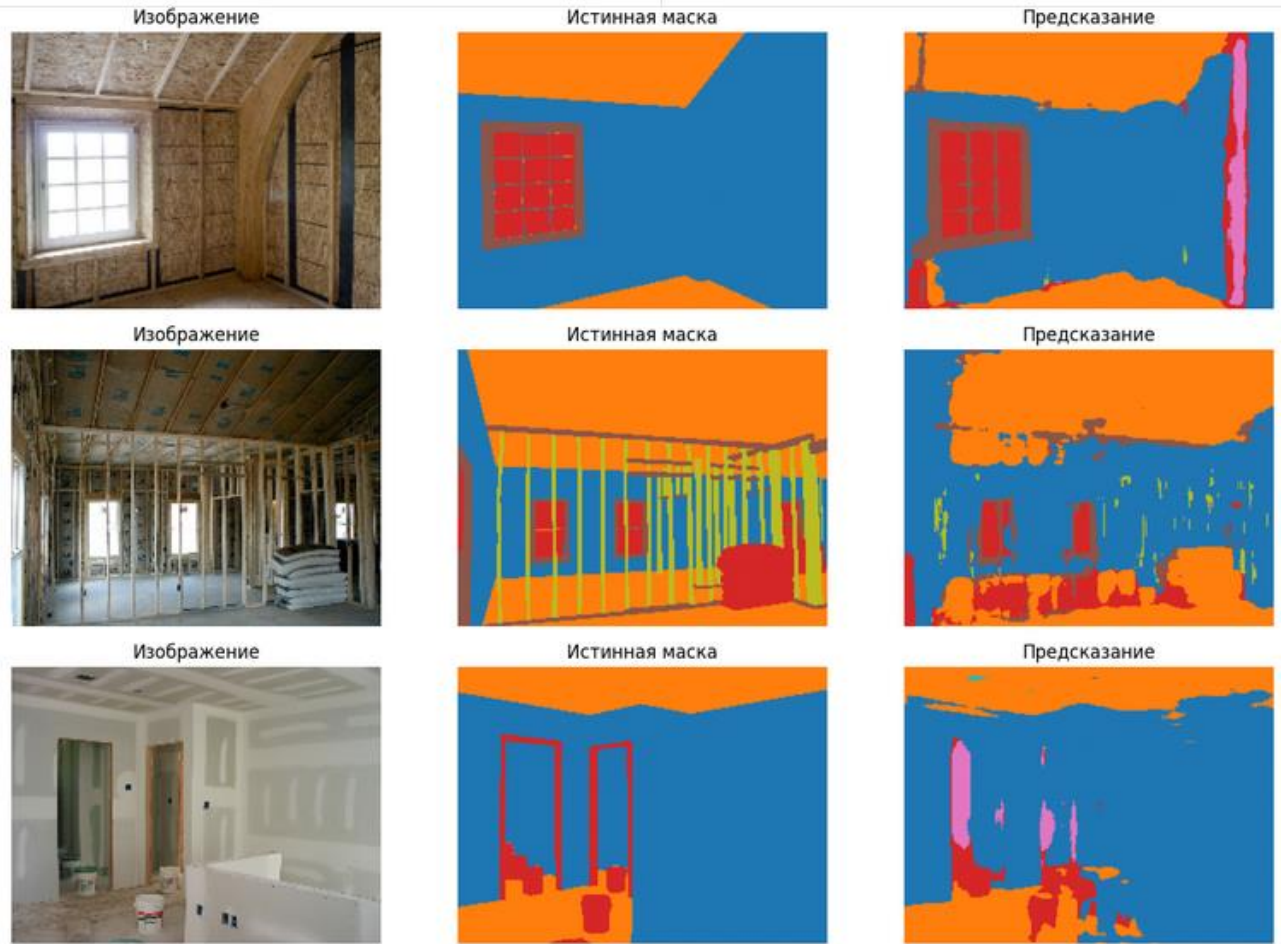


Рисунок 46 – Примеры предсказаний модели (Изображение, Истинная маска, Предсказание)

IoU по каждому классу на валидации:

0 FLOOR	IoU = 0.7886
1 CEILING	IoU = 0.6811
2 WALL	IoU = 0.3326
3 APERTURE	IoU = 0.2324
4 SUPPORT	IoU = 0.2122
5 INVENTORY	IoU = 0.0119
6 OTHER	IoU = 0.0630

Mean IoU (по присутствующим классам): 0.3203
8805

Рисунок 47 – IoU по каждому классу и Mean IoU на валидационной выборке

Анализ IoU показывает, что модель уверенно сегментирует крупные однородные области (FLOOR – 0.789, CEILING – 0.681), но испытывает затруднения с редкими классами (INVENTORY – 0.012, OTHER – 0.063) из-за сильного дисбаланса: FLOOR занимает 51% пикселей, тогда как INVENTORY – лишь 0.94%.

Задание Ultra Pro: На основе учебного ноутбука провести финальную подготовку данных. Изменить количество сегментирующих классов с 16 на 7 (те же категории, что и в Pro-варианте). Реализовать сегментацию строительных изображений на основе модели PSPNet (Pyramid Scene Parsing Network).

3.2. Особенности PSPNet.

PSPNet (Pyramid Scene Parsing Network, Zhao et al., 2017) отличается от U-Net принципиально иным подходом к захвату глобального контекста. Вместо симметричного энкодера-декодера с skip-соединениями PSPNet использует лёгкий CNN-backbone (4 уровня MaxPooling, снижение разрешения в 8 раз) и Pyramid Pooling Module (PPM) – набор AvgPool с различными размерами окон (1×1 , 2×2 , 3×3 , 6×6), после которых каждая ветвь масштабируется обратно до размера feature-карты и конкатенируется с исходной. Это позволяет модели одновременно учитывать локальные детали и глобальную сцену. Декодер PSPNet намеренно лёгкий – два conv_bn_act + финальный Conv2D – без skip-соединений к энкодеру.

3.3. Подготовка данных.

Импорт библиотек, загрузка датасета, оригинальных изображений и масок выполнены аналогично Pro-варианту. NUM_CLASSES=7, те же 7 классов: FLOOR, CEILING, WALL, APERTURE, SUPPORT, INVENTORY, OTHER.

```

# Импортируем модели keras: Model
from tensorflow.keras.models import Model

# Импортируем стандартные слои keras
from tensorflow.keras.layers import Input, Conv2DTranspose, concatenate, Activation
from tensorflow.keras.layers import MaxPooling2D, Conv2D, BatchNormalization, UpSampling2D

# Импортируем оптимизатор Adam
from tensorflow.keras.optimizers import Adam

# Импортируем модуль pyplot библиотеки matplotlib для построения графиков
import matplotlib.pyplot as plt

# Импортируем модуль image для работы с изображениями
from tensorflow.keras.preprocessing import image

# Импортируем библиотеку numpy
import numpy as np

# Импортируем метод деления выборки
from sklearn.model_selection import train_test_split

# Загрузка файлов по HTML ссылке
import gdown

# Для работы с файлами
import os

# Для генерации случайных чисел
import random

import time

# Импортируем модель Image для работы с изображениями
from PIL import Image

# очистка ОЗУ
import gc

```

Рисунок 48 – Импорт библиотек (Ultra Pro)

```

# Загрузка датасета из облака

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_256x192.zip', None, quiet=False)
#gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_512x384.zip', None, quiet=False)

!unzip -q 'construction_256x192.zip' # распаковываем архив

Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_256x192.zip
To: /content/construction_256x192.zip
100%|██████████| 214M/214M [00:23<00:00, 9.05MB/s]

```

Рисунок 49 – Загрузка датасета

Загрузка оригинальных изображений и подготовка данных с маппингом 16 → 7 классов аналогична заданию Pro. Реализована функция `load_images` для пакетного чтения в `uint8`, функция `load_segments_as_class7` для чтения масок через LUT. Итог: `x_train` (1900, 192, 256, 3) `uint8` – 281.2 МБ, `y_train` (1900, 192, 256) `int8` – 93.8 МБ.


```

# ШАГ 2. ЗАГРУЗКА ИЗОБРАЖЕНИЙ + 1b -> 7 КЛАССОВ
# =====
# Ноутбук Ultra Pro изначально не содержит ячеек чтения файлов,
# поэтому делаем это здесь.

import numpy as np
import gc
import tensorflow as tf

# --- Параметры ---
IMG_WIDTH = 192
IMG_HEIGHT = 256
TRAIN_DIRECTORY = 'train'
VAL_DIRECTORY = 'val'

NUM_CLASSES = 7
CLASS_NAMES = ['FLOOR', 'CEILING', 'WALL', 'APERTURE', 'SUPPORT', 'INVENTORY', 'OTHER']

# --- Загрузка картинок (как в учебном ноутбуке, но сразу в uint8 numpy) ---
def load_images(folder_subpath):
    """Читает все картинки из папки, возвращает uint8-массив (N, H, W, 3)."""
    files = sorted(os.listdir(folder_subpath))
    arrs = []
    for fn in files:
        img = image.load_img(os.path.join(folder_subpath, fn),
                              target_size=(IMG_WIDTH, IMG_HEIGHT))
        a = np.asarray(img, dtype=np.uint8)
        if a.ndim == 2: a = np.stack([a]*3, axis=-1)
        if a.shape[-1] == 4: a = a[:, :, :3]
        arrs.append(a)
    return np.stack(arrs, axis=0)

print('Загружаем train/original...')
t0 = time.time()
x_train = load_images(TRAIN_DIRECTORY + '/original')
print(f' x_train: {x_train.shape} {x_train.dtype} (за {time.time()-t0:.1f}c)')

print('Загружаем val/original...')
t0 = time.time()
x_val = load_images(VAL_DIRECTORY + '/original')
print(f' x_val: {x_val.shape} {x_val.dtype} (за {time.time()-t0:.1f}c)')

# --- Палитра 16 классов и маппинг 16 -> 7 ---
CLASS_COLORS_16 = [
    (0, 0, 0), # 0
    (128, 0, 0), # 1
    (0, 128, 0), # 2
    (128, 128, 0), # 3
    (0, 0, 128), # 4
    (128, 0, 128), # 5
    (0, 128, 128), # 6
    (128, 128, 128), # 7
    (64, 0, 0), # 8
    (192, 0, 0), # 9
    (64, 128, 0), # 10
    (192, 128, 0), # 11
    (64, 0, 128), # 12
    (192, 0, 128), # 13
    (64, 128, 128), # 14
    (192, 128, 128), # 15
]

```

Рисунок 50 – Загрузка изображений и подготовка данных: 16 → 7 классов
(часть 1)

```

# 7 целевых классов: FLOOR, CEILING, WALL, APERTURE, SUPPORT, INVENTORY, OTHER
MAP_16_TO_7 = np.array([
    0,          # 0
    1, 1,       # 1-2
    2, 2, 2,     # 3-5
    3, 3, 3,     # 6-8
    4, 4,       # 9-10
    5, 5,       # 11-12
    6, 6, 6,     # 13-15
], dtype=np.int8)

def build_lut(colors):
    """LUT по 24-битному ключу: RGB -> класс 0..15 (255 = неизвестный)."""
    lut = np.full(256 * 3, 255, dtype=np.uint8)
    for idx, (r, g, b) in enumerate(colors):
        lut[(r << 16) | (g << 8) | b] = idx
    return lut

def seg_rgb_to_class7(seg_rgb, lut):
    """RGB-маска -> карта 7 классов (H, W) int8."""
    s = np.asarray(seg_rgb, dtype=np.uint8)
    if s.ndim == 2:
        s = np.stack([s]*3, axis=-1)
    if s.shape[-1] == 4:
        s = s[..., :3]
    keys = ((s[...,0].astype(np.uint32) << 16) |
            (s[...,1].astype(np.uint32) << 8) |
            s[...,2].astype(np.uint32))
    cls16 = lut[keys]
    cls16 = np.where(cls16 == 255, 0, cls16)
    return MAP_16_TO_7[cls16]

def load_segments_as_class7(folder_subpath, lut):
    files = sorted(os.listdir(folder_subpath))
    out = []
    for fn in files:
        img = image.load_img(os.path.join(folder_subpath, fn),
                              target_size=(IMG_WIDTH, IMG_HEIGHT))
        out.append(seg_rgb_to_class7(img, lut))
    return np.stack(out, axis=0)

LUT = build_lut(CLASS_COLORS_16)

# Авто-проверка палитры на одной маске
probe_path = sorted(os.listdir(TRAIN_DIRECTORY + '/segment'))[0]
probe_img = image.load_img(os.path.join(TRAIN_DIRECTORY + '/segment', probe_path),
                             target_size=(IMG_WIDTH, IMG_HEIGHT))
probe = np.asarray(probe_img, dtype=np.uint8)
if probe.shape[-1] == 4: probe = probe[..., :3]
pk = ((probe[...,0].astype(np.uint32) << 16) |
       (probe[...,1].astype(np.uint32) << 8) |
       probe[...,2].astype(np.uint32))
known = (LUT[pk] != 255).mean()
print(f'\nИзвестных пикселей на первой маске: {known:.3f}')

```

Рисунок 51 – Подготовка данных: 16 → 7 классов (часть 2)

```

if known < 0.5:
    print(' >> Стандартная палитра не подошла, определяем 16 цветов автоматически по частоте...')
    keys_all = []
    for fn in sorted(os.listdir(TRAIN_DIRECTORY + '/segment')):
        img = image.load_img(os.path.join(TRAIN_DIRECTORY + '/segment', fn),
                                target_size=(IMG_WIDTH, IMG_HEIGHT))
        a = np.asarray(img, dtype=np.uint8)
        if a.shape[-1] == 4: a = a[..., :3]
        k = ((a[...,0].astype(np.uint32) << 16) |
              (a[...,1].astype(np.uint32) << 8) |
              a[...,2].astype(np.uint32))
        keys_all.append(k.ravel())
    keys_all = np.concatenate(keys_all)
    vals, counts = np.unique(keys_all, return_counts=True)
    top = vals[np.argsort(-counts)[:16]]
    print(f' уникальных цветов: {len(vals)}; берём топ-16.')
    LUT = np.full(256 ** 3, 255, dtype=np.uint8)
    for idx, k in enumerate(top):
        LUT[k] = idx
    del keys_all, vals, counts, top
    gc.collect()

print('\nЧитаем сегментации и сразу маппим 16->7...')
t0 = time.time()
y_train = load_segments_as_class7(TRAIN_DIRECTORY + '/segment', LUT)
print(f' y_train: {y_train.shape} {y_train.dtype} (за {time.time()-t0:.1f}c)')

t0 = time.time()
y_val = load_segments_as_class7(VAL_DIRECTORY + '/segment', LUT)
print(f' y_val: {y_val.shape} {y_val.dtype} (за {time.time()-t0:.1f}c)')

# Распределение классов
uniq, cnts = np.unique(y_train, return_counts=True)
total = cnts.sum()
print('\nРаспределение классов в y_train:')
for u, c in zip(uniq, cnts):
    name = CLASS_NAMES[u] if 0 <= u < len(CLASS_NAMES) else '?'
    print(f' {u} {name:<10s} {c:>10d} ({100.0*c/total:5.2f}%)')

print(f'\nПамять:')
print(f' X (uint8): {(x_train.nbytes + x_val.nbytes)/1024/1024:.1f} MB')
print(f' Y (int8): {(y_train.nbytes + y_val.nbytes)/1024/1024:.1f} MB')

gc.collect()

```

Рисунок 52 – Чтение масок и формирование Y

```

.. Загружаем train/original...
   x_train: (1900, 192, 256, 3) uint8 (за 0.7с)
Загружаем val/original...
   x_val: (100, 192, 256, 3) uint8 (за 0.0с)

Известных пикселей на первой маске: 0.000
>> Стандартная палитра не подошла, определяем 16 цветов автоматически по частоте...
уникальных цветов: 27; берём топ-16.

Читаем сегментации и сразу маппим 16->7...
y_train: (1900, 192, 256) int8 (за 1.4с)
y_val: (100, 192, 256) int8 (за 0.1с)

Распределение классов в y_train:
0 FLOOR      47798941 (51.18%)
1 CEILING    25679475 (27.50%)
2 WALL       13411319 (14.36%)
3 APERTURE    3783503 ( 4.05%)
4 SUPPORT     1483145 ( 1.59%)
5 INVENTORY   879546 ( 0.94%)
6 OTHER       352871 ( 0.38%)

Память:
X (uint8): 281.2 MB
Y (int8): 93.8 MB
0

```

Рисунок 53 – Результаты загрузки и распределение классов

3.4. Архитектура PSPNet.

Реализованы три компонента: функция `conv_bn_act` (Conv2D + BatchNorm + Activation), функция `backbone` (лёгкий 4-уровневый CNN с MaxPooling, снижение разрешения $\times 8$) и функция `pyramid_pooling_module` (PPM). PPM использует пулы размеров [1, 2, 3, 6], масштабирует каждую ветвь обратно до размера feature-карты через UpSampling2D с Resizing для выравнивания размеров, затем конкатенирует. Финальная функция `build_pspnet` собирает полную модель: `Input(uint8)  $\rightarrow$  Rescaling(1/255)  $\rightarrow$  backbone( $\times 8$ )  $\rightarrow$  PPM  $\rightarrow$  conv_bn_act  $\rightarrow$  Dropout(0.1)  $\rightarrow$  UpSampling( $\times 8$ )  $\rightarrow$  Resizing  $\rightarrow$  conv_bn_act  $\rightarrow$  Conv2D(7, softmax).` Модель скомпилирована с оптимизатором Adam ($lr=1e-3$) и потерями `sparse_categorical_crossentropy`.

```

# =====
# ШАГ 3. PSPNet (с нуля, без предобученного backbone)
# =====
# ВАЖНО: апсемплинг в Pyramid Pooling Module делаем через UpSampling2D
# (Keras-слой), а не tf.image.resize – последнее нельзя применять к
# символному KerasTensor внутри функциональной модели (Keras 3).

from tensorflow.keras.layers import (Input, Conv2D, BatchNormalization, Activation,
                                     MaxPooling2D, AveragePooling2D, UpSampling2D,
                                     concatenate, Rescaling, Dropout)

from tensorflow.keras.models import Model

from tensorflow.keras.optimizers import Adam

def conv_bn_act(x, filters, kernel_size=3, strides=1, activation='relu', name=None):
    x = Conv2D(filters, kernel_size, strides=strides, padding='same',
              kernel_initializer='he_normal', name=name)(x)
    x = BatchNormalization()(x)
    x = Activation(activation)(x)
    return x

def backbone(x, base=32):
    """Лёгкий 4-уровневый CNN-backbone. Понижение разрешения x8 (3 пулинга).

    Для входа 192x256: 192x256 -> 96x128 -> 48x64 -> 24x32
    Размеры feature-карты после backbone кратны 24 и делятся на нужные пулы.
    """
    x = conv_bn_act(x, base); x = conv_bn_act(x, base)
    x = MaxPooling2D()(x) # /2

    x = conv_bn_act(x, base*2); x = conv_bn_act(x, base*2)
    x = MaxPooling2D()(x) # /4

    x = conv_bn_act(x, base*4); x = conv_bn_act(x, base*4)
    x = MaxPooling2D()(x) # /8

    x = conv_bn_act(x, base*8); x = conv_bn_act(x, base*8) # без пулинга -> /8
    return x

def pyramid_pooling_module(x, feat_h, feat_w, pool_sizes=(1, 2, 3, 6),
                          filters_per_branch=None):
    """Pyramid Pooling Module из PSPNet (только Keras-слои).

    Для каждой ветви:
    AvgPool окном (ph, pw) -> 1x1 Conv -> BN -> ReLU -> UpSampling2D обратно.
    Затем всё конкатенируется с исходной feature-картой.
    feat_h, feat_w – известные (статические) размеры feature-карты.
    """
    C = x.shape[-1]
    if filters_per_branch is None:
        filters_per_branch = max(int(C // len(pool_sizes)), 16)

    branches = [x] # исходная карта остаётся в выходе
    for ps in pool_sizes:
        # окно пула: делим feature-карту на сетку ps x ps
        ph = max(feat_h // ps, 1)
        pw = max(feat_w // ps, 1)

        b = AveragePooling2D(pool_size=(ph, pw), strides=(ph, pw),
                           padding='same')(x)
        b = Conv2D(filters_per_branch, 1, padding='same',
                  kernel_initializer='he_normal')(b)
        b = BatchNormalization()(b)
        b = Activation('relu')(b)

        # размер после AvgPool (ceil из-за padding='same')
        out_h = -(-feat_h // ph) # ceil division

```

Рисунок 54 – Архитектура PSPNet: backbone и Pyramid Pooling Module (часть 1)

```

        # размер после AvgPool (ceil из-за padding='same')
        out_h = -(-feat_h // ph) # ceil division
        out_w = -(-feat_w // pw)
        # целочисленный апсемплинг обратно до (feat_h, feat_w).
        # Подбираем кратный коэффициент; при необходимости докроем размеры ниже.
        up_h = max(feat_h // out_h, 1)
        up_w = max(feat_w // out_w, 1)
        b = UpSampling2D(size=(up_h, up_w), interpolation='bilinear')(b)
        branches.append(b)

    # После UpSampling размеры веток могут отличаться от feat на 1-2 px
    # из-за округлений. Приводим все ветки к ровно (feat_h, feat_w)
    # обрезкой/падингом через Resizing-слой (Keras-слой, безопасен).
    from tensorflow.keras.layers import Resizing
    fixed = [branches[0]]
    for b in branches[1:]:
        fixed.append(Resizing(feat_h, feat_w, interpolation='bilinear')(b))
    return concatenate(fixed, axis=-1)

def build_pspnet(img_h, img_w, num_classes=NUM_CLASSES, base=32):
    """PSPNet: Input(uint8) -> Rescaling -> Backbone(/8) -> PPM -> Conv
    -> UpSampling x8 -> Softmax.
    """
    inp = Input(shape=(img_h, img_w, 3), dtype='uint8')
    x = Rescaling(1.0 / 255.0)(inp)

    feat = backbone(x, base=base) # (img_h/8, img_w/8, base*8)

    # статические размеры feature-карты
    feat_h = feat.shape[1]
    feat_w = feat.shape[2]

    ppm = pyramid_pooling_module(feat, feat_h, feat_w,
                                pool_sizes=(1, 2, 3, 6))

    x = conv_bn_act(ppm, base*4, kernel_size=3)
    x = Dropout(0.1)(x)

    # Поднимаем до исходного разрешения (x8) Keras-слоем
    x = UpSampling2D(size=(8, 8), interpolation='bilinear')(x)

    # На случай некрatности входного размера 8 - финально подгоняем
    from tensorflow.keras.layers import Resizing
    x = Resizing(img_h, img_w, interpolation='bilinear')(x)

    x = conv_bn_act(x, base*2, kernel_size=3)
    out = Conv2D(num_classes, 1, activation='softmax', name='seg_out')(x)

    model = Model(inputs=inp, outputs=out, name='pspnet')
    model.compile(
        optimizer=Adam(learning_rate=1e-3),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'],
    )
    return model

_, IMG_H, IMG_W, _ = x_train.shape
print(f'Размер входа PSPNet: ({IMG_H}, {IMG_W}, 3)')

tf.keras.backend.clear_session(); gc.collect()
pspnet = build_pspnet(IMG_H, IMG_W, base=32)
pspnet.summary()

```

Рисунок 55 – Архитектура PSPNet: Pyramid Pooling Module и build_pspnet
(часть 2)

• Размер входа PSPNet: (192, 256, 3)
Model: "pspnet"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 192, 256, 3)	0	-
rescaling (Rescaling)	(None, 192, 256, 3)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 192, 256, 32)	896	rescaling[0][0]
batch_normalization (BatchNormalization)	(None, 192, 256, 32)	128	conv2d[0][0]
activation (Activation)	(None, 192, 256, 32)	0	batch_normalizat...
conv2d_1 (Conv2D)	(None, 192, 256, 32)	9,248	activation[0][0]
batch_normalization (BatchNormalization)	(None, 192, 256, 32)	128	conv2d_1[0][0]
activation_1 (Activation)	(None, 192, 256, 32)	0	batch_normalizat...
max_pooling2d (MaxPooling2D)	(None, 96, 128, 32)	0	activation_1[0][...
conv2d_2 (Conv2D)	(None, 96, 128, 64)	18,496	max_pooling2d[0][...
batch_normalization (BatchNormalization)	(None, 96, 128, 64)	256	conv2d_2[0][0]
activation_2 (Activation)	(None, 96, 128, 64)	0	batch_normalizat...
conv2d_3 (Conv2D)	(None, 96, 128, 64)	36,928	activation_2[0][...
batch_normalization (BatchNormalization)	(None, 96, 128, 64)	256	conv2d_3[0][0]
activation_3 (Activation)	(None, 96, 128, 64)	0	batch_normalizat...
max_pooling2d_1 (MaxPooling2D)	(None, 48, 64, 64)	0	activation_3[0][...
conv2d_4 (Conv2D)	(None, 48, 64, 128)	73,856	max_pooling2d_1[...
batch_normalization (BatchNormalization)	(None, 48, 64, 128)	512	conv2d_4[0][0]
activation_4 (Activation)	(None, 48, 64, 128)	0	batch_normalizat...

Рисунок 56 – Краткая сводка архитектуры модели PSPNet

3.5. Обучение PSPNet.

Модель обучалась с EPOCHS=60 и BATCH_SIZE=8. Коллбэки: EarlyStopping (patience=12, restore_best_weights=True), ReduceLROnPlateau (factor=0.5, patience=4, min_lr=1e-6), ModelCheckpoint (best_pspnet.weights.h5). Обучение остановилось на эпохе 28 (EarlyStopping), лучшие веса восстановлены с эпохи 16. Финальные метрики: train_acc=0.9066, val_acc=0.7260, лучшая val_acc=0.7260, лучший val_loss=0.8649.


```

# =====
# ЦАГ 4. ОБУЧЕНИЕ PSPNet
# =====
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

EPOCHS = 60
BATCH_SIZE = 8

# tf.data dataset с prefetch – без копирования больших тензоров
def make_ds(x, y, batch, shuffle):
    ds = tf.data.Dataset.from_tensor_slices((x, y))
    if shuffle:
        ds = ds.shuffle(buffer_size=min(512, len(x)), reshuffle_each_iteration=True)
    return ds.batch(batch).prefetch(tf.data.AUTOTUNE)

train_ds = make_ds(x_train, y_train, BATCH_SIZE, shuffle=True)
val_ds = make_ds(x_val, y_val, BATCH_SIZE, shuffle=False)

callbacks = [
    EarlyStopping(monitor='val_loss', patience=12,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4,
                     min_lr=1e-6, verbose=1),
    ModelCheckpoint('best_pspnet_weights.h5', monitor='val_loss',
                   save_best_only=True, save_weights_only=True, verbose=0),
]

tf.keras.backend.clear_session(); gc.collect()
pspnet = build_pspnet(IMG_H, IMG_W, base=32)

history = pspnet.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=2,
)

print('\nОбучение завершено.')
print(f'Финальная train_acc: {history.history['accuracy'][-1]:.4f}')
print(f'Финальная val_acc: {history.history['val_accuracy'][-1]:.4f}')
print(f'Лучшая val_acc: {max(history.history['val_accuracy']):.4f}')
print(f'Финальная val_loss: {history.history['val_loss'][-1]:.4f}')
print(f'Лучшая val_loss: {min(history.history['val_loss']):.4f}')

```

Рисунок 57 – Код обучения PSPNet (коллбэки, model.fit)

```

Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
238/238 - 35s - 146ms/step - accuracy: 0.8204 - loss: 0.5058 - val_accuracy: 0.6674 - val_loss: 0.9684 - learning_rate: 0.0010
Epoch 11/60
238/238 - 35s - 147ms/step - accuracy: 0.8345 - loss: 0.4687 - val_accuracy: 0.7000 - val_loss: 0.8852 - learning_rate: 5.0000e-04
Epoch 12/60
238/238 - 35s - 148ms/step - accuracy: 0.8428 - loss: 0.4484 - val_accuracy: 0.6983 - val_loss: 0.8673 - learning_rate: 5.0000e-04
Epoch 13/60
238/238 - 35s - 147ms/step - accuracy: 0.8491 - loss: 0.4238 - val_accuracy: 0.7127 - val_loss: 0.8827 - learning_rate: 5.0000e-04
Epoch 14/60
238/238 - 35s - 147ms/step - accuracy: 0.8510 - loss: 0.4177 - val_accuracy: 0.6652 - val_loss: 0.9251 - learning_rate: 5.0000e-04
Epoch 15/60
238/238 - 35s - 147ms/step - accuracy: 0.8552 - loss: 0.4077 - val_accuracy: 0.6972 - val_loss: 0.8899 - learning_rate: 5.0000e-04
Epoch 16/60
238/238 - 35s - 149ms/step - accuracy: 0.8586 - loss: 0.3955 - val_accuracy: 0.7093 - val_loss: 0.8649 - learning_rate: 5.0000e-04
Epoch 17/60
238/238 - 35s - 147ms/step - accuracy: 0.8623 - loss: 0.3851 - val_accuracy: 0.6861 - val_loss: 0.9893 - learning_rate: 5.0000e-04
Epoch 18/60
238/238 - 35s - 147ms/step - accuracy: 0.8658 - loss: 0.3748 - val_accuracy: 0.6801 - val_loss: 0.9550 - learning_rate: 5.0000e-04
Epoch 19/60
238/238 - 35s - 147ms/step - accuracy: 0.8712 - loss: 0.3603 - val_accuracy: 0.6689 - val_loss: 0.9716 - learning_rate: 5.0000e-04
Epoch 20/60

Epoch 20: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
238/238 - 35s - 147ms/step - accuracy: 0.8727 - loss: 0.3550 - val_accuracy: 0.6884 - val_loss: 0.9505 - learning_rate: 5.0000e-04
Epoch 21/60
238/238 - 35s - 147ms/step - accuracy: 0.8856 - loss: 0.3179 - val_accuracy: 0.7006 - val_loss: 0.9668 - learning_rate: 2.5000e-04
Epoch 22/60
238/238 - 35s - 147ms/step - accuracy: 0.8910 - loss: 0.3049 - val_accuracy: 0.7078 - val_loss: 0.8701 - learning_rate: 2.5000e-04
Epoch 23/60
238/238 - 35s - 147ms/step - accuracy: 0.8927 - loss: 0.2987 - val_accuracy: 0.7038 - val_loss: 0.8951 - learning_rate: 2.5000e-04
Epoch 24/60

Epoch 24: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
238/238 - 35s - 147ms/step - accuracy: 0.8944 - loss: 0.2944 - val_accuracy: 0.7185 - val_loss: 0.8875 - learning_rate: 2.5000e-04
Epoch 25/60
238/238 - 35s - 147ms/step - accuracy: 0.9011 - loss: 0.2752 - val_accuracy: 0.7248 - val_loss: 0.8740 - learning_rate: 1.2500e-04
Epoch 26/60
238/238 - 35s - 147ms/step - accuracy: 0.9033 - loss: 0.2697 - val_accuracy: 0.7093 - val_loss: 0.8693 - learning_rate: 1.2500e-04
Epoch 27/60
238/238 - 35s - 147ms/step - accuracy: 0.9059 - loss: 0.2627 - val_accuracy: 0.7104 - val_loss: 0.8966 - learning_rate: 1.2500e-04
Epoch 28/60

Epoch 28: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
238/238 - 35s - 147ms/step - accuracy: 0.9066 - loss: 0.2595 - val_accuracy: 0.7260 - val_loss: 0.8987 - learning_rate: 1.2500e-04
Epoch 28: early stopping
Restoring model weights from the end of the best epoch: 16.

Обучение завершено.
Финальная train_acc: 0.9066
Финальная val_acc: 0.7260
Лучшая val_acc: 0.7260
Финальная val_loss: 0.8987
Лучшая val_loss: 0.8649

```

Рисунок 58 – Ход обучения PSPNet и итоговые метрики

3.6. Графики обучения.

Кривая ассигасу на обучающей выборке выросла до ~ 0.91 , на валидационной – стабилизировалась около $0.72\text{--}0.73$. Кривая loss на обучающей снизилась до ~ 0.26 , на валидационной – удерживалась около $0.87\text{--}1.2$ с характерными выбросами на ранних эпохах.

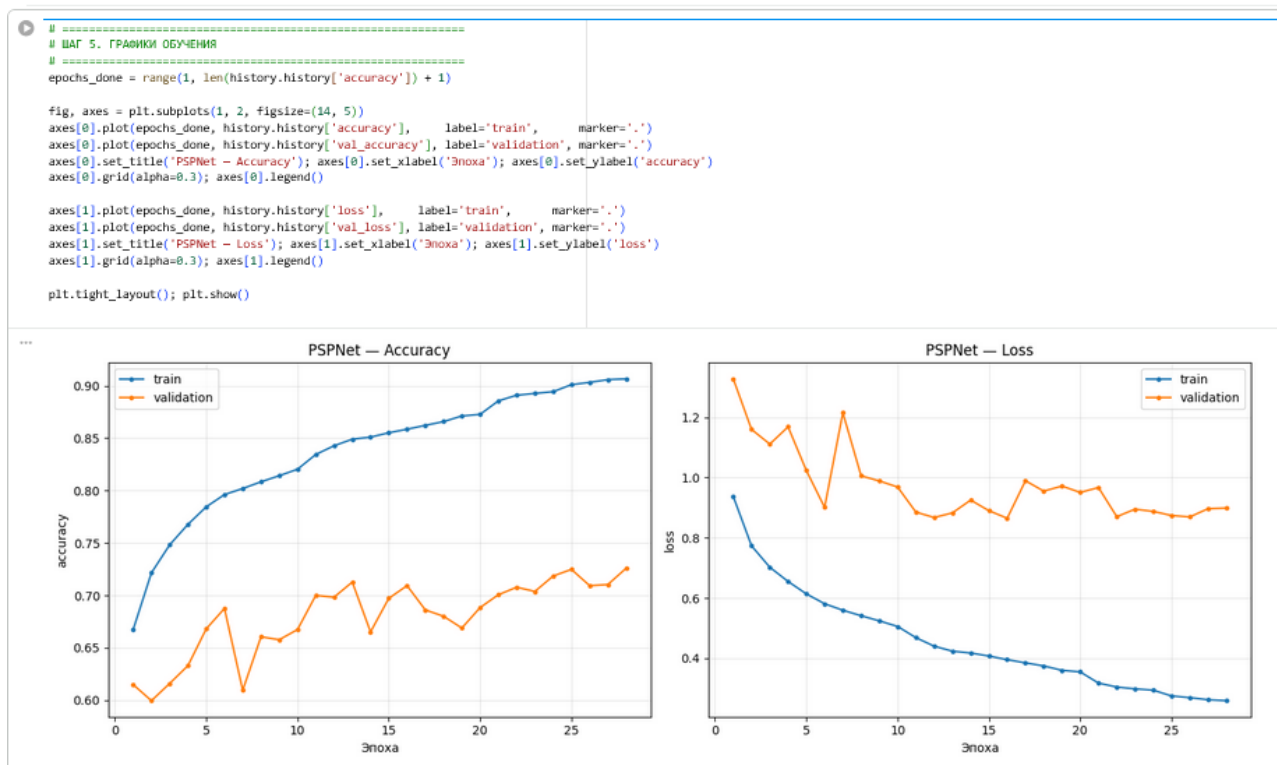


Рисунок 59 – Кривые PSPNet – Accuracy и Loss по эпохам

3.7. Визуализация предсказаний и метрика IoU.

Значения IoU по классам: FLOOR – 0.6679, CEILING – 0.6509, WALL – 0.3150, APERTURE – 0.1930, SUPPORT – 0.1825, INVENTORY – 0.0420, OTHER – 0.0494. Mean IoU – 0.3001. По сравнению с канонической U-Net (Mean IoU=0.3203) PSPNet показал несколько более низкий результат, однако обучился быстрее (28 эпох vs 42). PSPNet лучше справился с классом CEILING (0.651 vs 0.681 – сопоставимо), но хуже с FLOOR (0.668 vs 0.789).

```

# --- Примеры с валидации ---
n_show = 5
idxs = np.random.choice(len(x_val), n_show, replace=False)
sample_x = x_val[idxs]
preds = pspnet.predict(sample_x, batch_size=BATCH_SIZE, verbose=0)
pred_classes = np.argmax(preds, axis=-1)
true_classes = y_val[idxs]

fig, axes = plt.subplots(n_show, 3, figsize=(13, 3.0 * n_show))
for i in range(n_show):
    axes[i, 0].imshow(sample_x[i]); axes[i, 0].set_title('Изображение'); axes[i, 0].axis('off')
    axes[i, 1].imshow(true_classes[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1)
    axes[i, 1].set_title('Истинная маска'); axes[i, 1].axis('off')
    axes[i, 2].imshow(pred_classes[i], cmap='tab10', vmin=0, vmax=NUM_CLASSES-1)
    axes[i, 2].set_title('Предсказание (PSPNet)'); axes[i, 2].axis('off')
plt.tight_layout(); plt.show()

# --- IoU по каждому классу на полной валидации (батчами, чтобы не съест память) ---
print('\nIoU по каждому классу на валидации:')
batch = 16
val_pred_all = np.empty_like(y_val)
for s in range(0, len(x_val), batch):
    p = pspnet.predict(x_val[s:s+batch], verbose=0)
    val_pred_all[s:s+batch] = np.argmax(p, axis=-1).astype(np.int8)

ious = []
for c in range(NUM_CLASSES):
    inter = np.logical_and(y_val == c, val_pred_all == c).sum()
    union = np.logical_or(y_val == c, val_pred_all == c).sum()
    iou = inter / union if union > 0 else float('nan')
    ious.append(iou)
    print(f' {c} {CLASS_NAMES[c]:<10s} IoU = {iou:.4f}')

valid = [v for v in ious if not np.isnan(v)]
print(f'\nMean IoU (по присутствующим классам): {np.mean(valid):.4f}')

del preds, sample_x, val_pred_all
gc.collect()

```

Рисунок 60 – Код визуализации предсказаний и IoU (PSPNet)



Рисунок 61 – Примеры предсказаний PSPNet

```
IoU по каждому классу на валидации:
0 FLOOR      IoU = 0.6679
1 CEILING    IoU = 0.6589
2 WALL       IoU = 0.3158
3 APERTURE    IoU = 0.1938
4 SUPPORT    IoU = 0.1825
5 INVENTORY  IoU = 0.8428
6 OTHER      IoU = 0.8494

Mean IoU (по присутствующим классам): 0.3881
8788
```

Рисунок 62 – IoU по классам и Mean IoU (PSPNet)

Выводы: в ходе лабораторной работы выполнены все три задания по сегментации строительных изображений с использованием архитектуры U-Net.